COMPLETE STUDY NOTES · 22 CHAPTERS · BEGINNER TO ADVANCED

Azure Cloud & DevOps — Master Study Guide

A complete, single-source reference to Microsoft Azure & DevOps — from cloud fundamentals and Linux basics through compute, networking, storage, identity, databases, serverless, CI/CD, Kubernetes, and Infrastructure as Code. Every theory note, diagram description, command table, and scenario-based interview Q&A from the source material is preserved in full, organized by the document's own service categories.

Cloud Fundamentals · Linux Basics · Compute Services · Storage Services · Networking · Object Storage (Blob) · Identity & Access · Databases · Serverless · CI/CD Pipelines · Orchestration (AKS) · Infrastructure as Code

INDEX · JUMP TO A CHAPTER

OV

1. Introduction to Cloud Computing

Cloud computing models, CapEx vs OpEx, shared responsibility, and costing.

Cloud Fundamentals

VIRT

2. Virtualization

Hypervisors, Type 1 vs Type 2, and how virtualization enables the cloud.

Cloud Fundamentals

LNX

3. Linux Basics

The Linux filesystem, permissions, processes, users, and networking commands.

Linux Basics

VM

4. Azure Virtual Machines — Overview & Instance Launch

Azure Virtual Machines — images, sizes, and launching your first instance.

Compute Services

VMOP

5. Azure VM Instance Management

VM lifecycle management — SSH keys, NSGs, disks, snapshots, and metadata.

Compute Services

LB

6. Azure Load Balancer

Distributing traffic — Load Balancer, Application Gateway, Traffic Manager, Front Door.

Compute Services

MON

7. Billing and Monitoring

Cost management and Azure Monitor — budgets, metrics, alerts, KQL.

Compute Services

SCALE

8. Azure Auto Scaling

Virtual Machine Scale Sets and reactive/scheduled/predictive autoscaling.

Compute Services

DISK

9. Storage Services — Managed Disks & Azure Files

Managed Disks and Azure Files — block storage vs shared file storage.

Storage Services

VNET

10. Networking — Virtual Network (VNET)

Virtual Network fundamentals — subnets, CIDR, public vs private IPs.

Networking

VNETC

11. Networking — VNET Controls

NSGs as ACLs, route tables, VPN/ExpressRoute gateways, hub-and-spoke.

Networking

BLOB

12. Object Storage — Azure Blob Storage

Azure Blob Storage fundamentals — containers, tiers, versioning, SAS.

Object Storage (Blob)

BLOBA

13. Object Storage — Blob Advanced

Partitioning, AzCopy, replication (LRS/ZRS/GRS), lifecycle policies, Key Vault.

Object Storage (Blob)

BLOBX

14. Object Storage — Cross-Account Blob Access

Cross-account and cross-subscription Blob access patterns.

Object Storage (Blob)

ID

15. Identity & Access — Microsoft Entra ID (Azure AD)

Microsoft Entra ID — MFA, RBAC, managed identities, auditing.

Identity & Access

SQL

16. Databases — Azure SQL Database

Azure SQL Database — PaaS vs IaaS, connection proxy modes, Redis cache.

Databases

SQLOP

17. Databases — Azure SQL Operations

Connecting to Azure SQL, running queries, and read replicas.

Databases

APP

18. Serverless — Azure App Service & Functions

Azure App Service and Functions — PaaS web apps and FaaS.

Serverless

APPI

19. Serverless — App Service Integrations & Limitations

Function limitations and integrations — SQL, Cosmos DB, API Management.

Serverless

CICD

20. CI/CD Pipelines — Azure DevOps

Azure DevOps Pipelines — stages, YAML, deployment strategies.

CI/CD Pipelines

AKS

21. Orchestration — Azure Kubernetes Service (AKS)

Azure Kubernetes Service — control plane, node pools, autoscaling.

Orchestration (AKS)

TF

22. Infrastructure as Code — Terraform with Azure

Terraform with the azurerm provider — state, modules, CI/CD workflow.

Infrastructure as Code

OV

CHAPTER 01 · CLOUD FUNDAMENTALS

Introduction to Cloud Computing

Cloud computing models, CapEx vs OpEx, shared responsibility, and costing.

Cloud Computing is the on-demand delivery of IT resources — compute, storage, databases, networking, and software — over the internet, with pay-as-you-go pricing. Instead of buying and maintaining physical servers in a data center, an organisation rents exactly the capacity it needs from a provider such as Microsoft Azure, AWS, or Google Cloud, and scales that capacity up or down in minutes.

💡 Layman Explanation

Think of cloud computing like an electricity grid instead of running your own generator. Before the grid existed, every factory needed its own power plant — expensive to build, expensive to maintain, and wasted if under-used. The grid let anyone plug in and pay only for the electricity consumed.

Cloud computing does the same for computing power: Microsoft (or AWS, Google) already built massive data centers around the world. You just "plug in" over the internet, use exactly what you need, and get billed only for that usage.

1.1 Why Cloud Computing?

📝 Theory & Key Points

- Before the cloud, companies practiced **CapEx** (Capital Expenditure) IT — buying racks of physical servers upfront, provisioning for peak load that might only occur a few days a year, and waiting weeks for procurement and installation.
- Cloud computing shifts this to **OpEx** (Operational Expenditure) — you rent resources monthly/hourly, so cash isn't locked into idle hardware.
- Elimination of guesswork in capacity planning: if traffic spikes 10x during a sale, Auto Scaling adds servers automatically; when it settles, servers are removed.
- Global reach in minutes: an Azure Virtual Machine can be deployed in India, the US, or Europe within seconds, without physically shipping any hardware.
- Increased speed and agility: new environments (dev/test/prod) that used to take weeks to rack and cable now take minutes via the Azure Portal, CLI, or Terraform.
- Focus on business value, not "undifferentiated heavy lifting" — patching hypervisors, replacing failed disks, and cooling data centers becomes Microsoft's job, not yours.

1.2 Benefits of Cloud Computing

BENEFIT	WHAT IT MEANS IN PRACTICE
Trade CapEx for OpEx	No upfront hardware purchase; pay hourly/monthly like a utility bill.
Economies of scale	Microsoft buys hardware at massive scale across millions of customers, so per-unit cost is lower than what any single company could achieve alone.
Stop guessing capacity	Scale to actual demand instead of provisioning for a hypothetical peak.
Increase speed & agility	New resources are self-service and available in minutes, encouraging experimentation.
Stop spending on data centers	No racking, stacking, cooling, or physical security to manage.
Go global in minutes	Deploy an application to multiple Azure regions worldwide with a few clicks or one Terraform apply.

1.3 Types of Cloud Computing (Deployment Models)

MODEL	DESCRIPTION	EXAMPLE USE CASE
Public Cloud	Resources owned and operated by a third-party provider (Azure, AWS, GCP) and shared across many tenants via strict isolation.	Startups, SaaS products, most modern workloads.
Private Cloud	Cloud-like resources dedicated to a single organisation, hosted either on-premises or by a provider.	Banks/governments with strict data residency or compliance needs.
Hybrid Cloud	A mix of on-premises/private infrastructure connected to public cloud, often via a VPN or ExpressRoute/Direct Connect.	Bursting to the cloud during peak load while keeping sensitive data on-prem.

1.4 Cloud Service Models

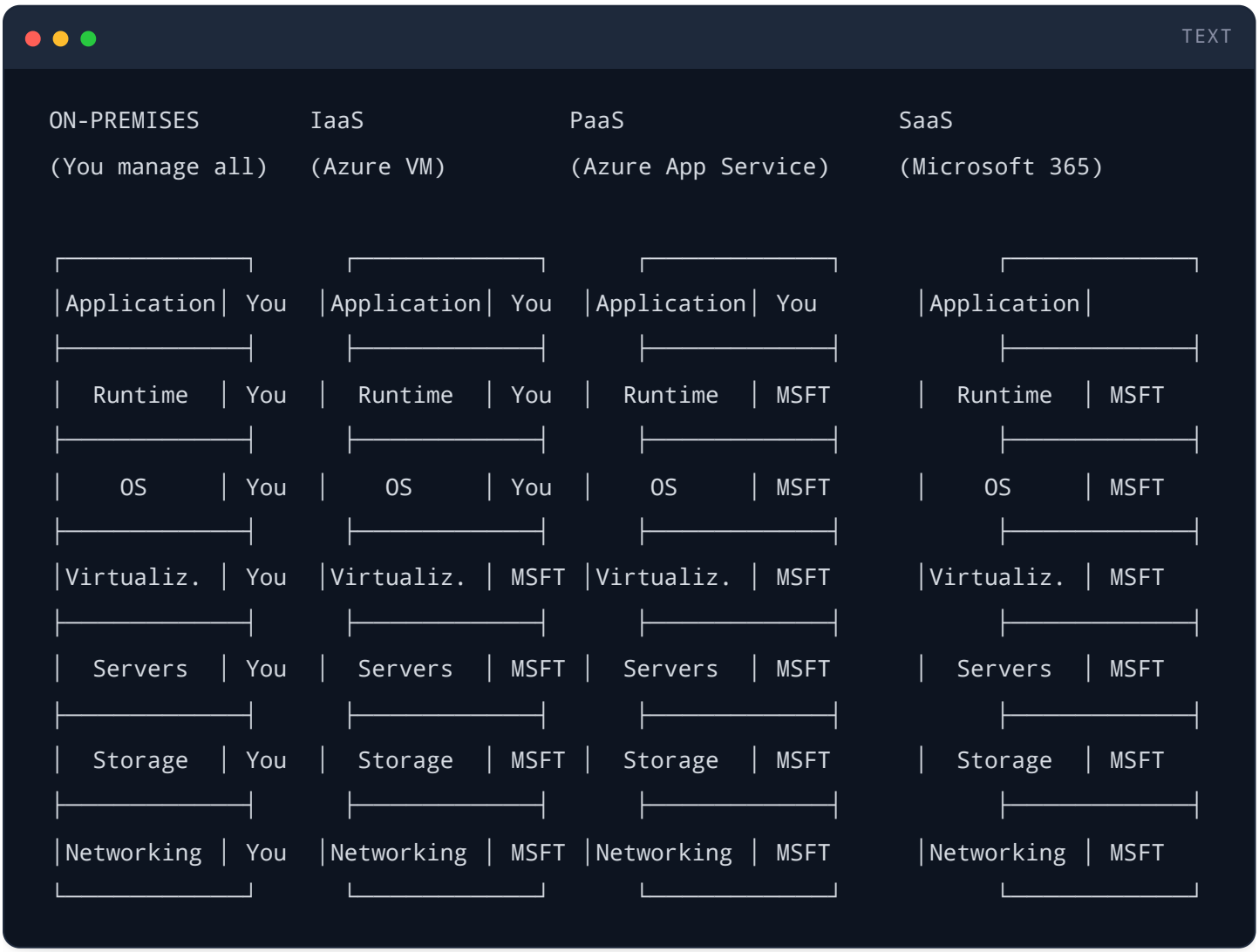


Fig 1: The Cloud Service Model Stack — What You Manage vs What the Provider Manages

MODEL	YOU MANAGE	PROVIDER MANAGES	AZURE EXAMPLE
IaaS (Infrastructure as a Service)	OS, runtime, application, data, patching, security groups	Physical servers, hypervisor, storage hardware, networking hardware	Azure Virtual Machines, Managed Disks, VNET
PaaS (Platform as a Service)	Application code and data only	OS, runtime, patching, scaling infrastructure, load balancing	Azure App Service, Azure Functions, Azure SQL Database
SaaS (Software as a Service)	Just your data/configuration inside the app	Everything — the entire application stack	Microsoft 365, Dynamics 365, Outlook

1.5 Scaling in Cloud Computing

📝 Theory & Key Points

- **Vertical Scaling (Scale Up/Down):** increasing or decreasing the size (CPU/RAM) of a single instance — e.g. resizing a VM from Standard_B2s to Standard_D4s_v5. Has a ceiling (the largest VM size) and usually requires a restart.
- **Horizontal Scaling (Scale Out/In):** adding or removing more instances behind a load balancer. This is the cloud-native pattern and has virtually no ceiling — this is what Azure Virtual Machine Scale Sets and AKS do automatically.
- Horizontal scaling is generally preferred in the cloud because it improves both capacity AND availability (more instances = more fault tolerance), whereas vertical scaling is a single point of failure.
- Auto Scaling policies typically trigger on metrics such as CPU %, memory, queue length, or custom Application Insights metrics.

1.6 Cloud Computing Issues & Challenges

📝 Theory & Key Points

- **Vendor lock-in:** deep use of provider-specific services (e.g. proprietary PaaS features) makes migrating to another cloud costly. Mitigated with IaC (Terraform) and containerisation.
- **Cost management:** the pay-as-you-go model can spiral without governance — orphaned disks, oversized VMs, and forgotten test resources are common cost leaks. Azure Cost Management + Budgets + tagging policies address this.
- **Data security & compliance:** data residency laws (GDPR, RBI guidelines, HIPAA) may require data to physically stay within a region.
- **Shared Responsibility Model:** the provider secures the cloud (physical data centers, host infrastructure); the customer secures what's IN the cloud (identity, data, network configuration, OS patching for IaaS).
- **Downtime/outage risk:** even hyperscale providers have regional outages — multi-region and multi-AZ architectures reduce this risk.

1.7 Security in the Cloud

📝 Theory & Key Points

- Security in Azure follows the Shared Responsibility Model: physical security and host patching is Microsoft's job; identity, access control (RBAC), network security groups, encryption keys, and data classification are the customer's job.
- Defense in depth: identity (Entra ID + MFA) → network (NSGs, firewalls) → compute (patching, endpoint protection) → application (secure coding) → data (encryption at rest and in transit).
- Principle of Least Privilege: grant users/service principals only the exact roles they need via Azure RBAC — never use the Owner role for daily operations.
- Microsoft Defender for Cloud provides continuous security posture assessment and threat detection across Azure resources.

1.8 Cloud Costing Models

PRICING MODEL	DESCRIPTION	BEST FOR
Pay-As-You-Go	Billed per second/hour of actual usage, no commitment.	Unpredictable or short-term workloads, dev/test.
Reserved Instances / Savings Plan	Commit to 1 or 3 years of usage for a discount of up to ~60-72%.	Steady-state, predictable production workloads.
Spot Pricing	Bid for unused Azure capacity at steep discounts (up to 90%); can be reclaimed by Azure with short notice.	Fault-tolerant batch jobs, CI runners, stateless workers.
Hybrid Benefit	Reuse existing on-prem Windows Server / SQL Server licenses in Azure to reduce cost.	Enterprises migrating existing licensed workloads.

🎯 Scenario-Based Interview Questions

Q: Your startup runs a food-delivery app. Traffic is 10x higher on Friday/Saturday nights and near-zero at 4 AM. How would you architect this cost-effectively in Azure, and which scaling approach fits best? Use horizontal scaling via Azure Virtual Machine Scale Sets (or AKS with the Horizontal Pod Autoscaler) fronted by an Azure Load Balancer / Application Gateway. Configure an autoscale rule on CPU % or request-queue-length so instances scale out automatically before Friday peak and scale back in overnight. Combine Pay-As-You-Go for the elastic layer with Reserved Instances for the small steady baseline capacity that's always running, and use Azure Cost Management budgets/alerts to catch runaway scale-out. This avoids provisioning for peak 24/7 (wasteful CapEx-style thinking) while still meeting weekend demand.

Q: A finance company asks: 'If Microsoft manages the data center security, why do we still need our own security team for Azure?' Because of the Shared Responsibility Model — Microsoft secures the physical facility, host hypervisor, and underlying network hardware ('security OF the cloud'), but the customer is fully responsible for identity and access management, network security group rules, OS-level patching on IaaS VMs, data encryption/classification, and application-level security ('security IN the cloud'). A misconfigured NSG rule or an over-privileged RBAC assignment is entirely the customer's liability, regardless of how secure Microsoft's data center is.

VIRT

CHAPTER 02 · CLOUD FUNDAMENTALS

Virtualization

Hypervisors, Type 1 vs Type 2, and how virtualization enables the cloud.

Virtualization is the foundational technology that makes cloud computing possible. It is the process of creating a software-based (virtual) version of something physical — a server, storage device, or network — so that one physical machine can be divided into many independent, isolated virtual machines (VMs), each believing it has its own dedicated hardware.

💡 Layman Explanation

Imagine one large apartment building (the physical server). Virtualization is like dividing that building into separate flats (virtual machines), each with its own locked door, its own kitchen, and its own address — even though they all share the same building foundation, plumbing, and electricity (the physical CPU, RAM, disk).

Each tenant (VM) has no idea other tenants exist. They can install their own furniture (operating system) and never interfere with the neighbours.

2.1 Virtualization and Cloud Computing

📝 Theory & Key Points

- Without virtualization, cloud providers would have to give each customer a dedicated physical server — extremely wasteful, since most workloads use only a fraction of a server's CPU/RAM.
- A **Hypervisor** sits directly on physical hardware (or on a host OS) and allows multiple guest VMs to run simultaneously, each with isolated CPU, memory, disk, and network allocations.
- Azure's compute fabric runs on Microsoft's own hypervisor, Hyper-V, at massive scale across its global data centers.
- Virtualization is what enables elasticity: spinning up a new Azure VM doesn't require new physical hardware — it just carves out a new isolated slice of an already-running physical host.

2.2 Types of Virtualization

TYPE	WHAT IS VIRTUALIZED	AZURE EXAMPLE
Server / Compute Virtualization	One physical server divided into multiple VMs	Azure Virtual Machines
Storage Virtualization	Physical disks pooled and presented as flexible virtual disks/volumes	Azure Managed Disks, Storage Accounts
Network Virtualization	Physical network abstracted into software-defined virtual networks	Azure Virtual Network (VNET), NSGs
OS-level Virtualization (Containers)	The OS kernel is shared, but processes are isolated into lightweight containers (no separate guest OS)	Azure Container Instances, AKS
Desktop Virtualization	A full desktop OS/session delivered remotely	Azure Virtual Desktop

2.3 Virtualization Terminologies

TERM	MEANING
Host Machine	The physical server that runs the hypervisor and hosts one or more VMs.
Guest Machine (VM)	The virtual machine running on top of the host, with its own OS and virtual hardware.
Hypervisor / VMM	Virtual Machine Monitor — the software layer that creates, runs, and manages VMs, allocating physical CPU/RAM/disk among them.
Virtual CPU (vCPU)	A logical CPU core presented to a guest VM, mapped by the hypervisor onto physical CPU time-slices.
Snapshot	A point-in-time saved state of a VM's disk (and optionally memory) used for backup or quick rollback.
Live Migration	Moving a running VM from one physical host to another with zero (or near-zero) downtime — used by Azure for host maintenance.

2.4 Hypervisor Concepts — Type 1 vs Type 2

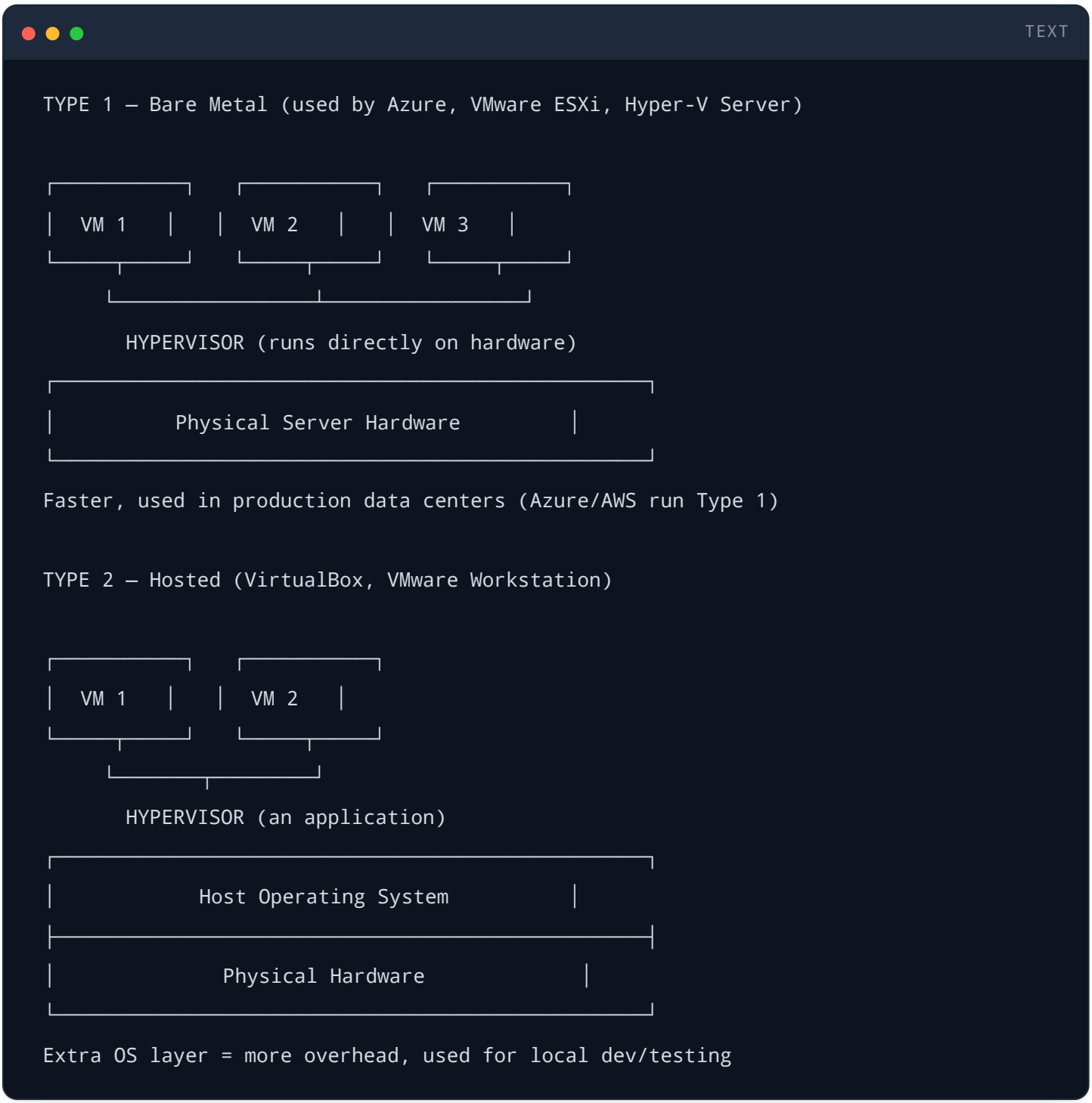


Fig 2: Type 1 (Bare-Metal) vs Type 2 (Hosted) Hypervisor

2.5 Benefits of Virtualization

📝 Theory & Key Points

- **Higher hardware utilisation:** a single physical server that would sit at 10% utilisation running one workload can run dozens of VMs, each using a slice of that capacity.
- **Isolation:** a crash or security compromise inside one VM does not affect neighbouring VMs on the same host.
- **Faster provisioning:** a new VM is created from a template/image in minutes rather than the days/weeks needed to rack physical hardware.
- **Portability:** a VM is just a set of files (disk image + configuration) and can be moved, cloned, or backed up far more easily than physical hardware.
- **Cost efficiency:** underpins the entire cloud pay-as-you-go economic model — this is the mechanism that lets Azure sell fractional slices of a server.

2.6 Major Hypervisor Vendors

VENDOR / PRODUCT	TYPE	NOTES
Microsoft Hyper-V	Type 1	Powers Azure's own compute fabric and Windows Server virtualization.
VMware ESXi / vSphere	Type 1	Dominant in enterprise on-prem data centers; Azure VMware Solution lets you run vSphere natively inside Azure.
KVM (Kernel-based VM)	Type 1 (Linux kernel module)	Open-source; underlies AWS Nitro and many OpenStack deployments.
Oracle VirtualBox	Type 2	Free, common for local development and learning labs.

🎯 Scenario-Based Interview Questions

Q: A colleague asks: 'If Azure VMs are already virtual, why does Azure also offer containers (AKS/ACI)? Aren't they the same thing?' They solve isolation at different layers. A VM virtualizes the entire hardware stack, including a full guest OS kernel — heavier (GBs, boots in ~1 minute) but strongly isolated, ideal when you need a different OS or kernel-level customisation. A container shares the host's OS kernel and only virtualizes the process/filesystem/network namespace — much lighter (MBs, starts in ~1 second) and ideal for packaging and scaling application code consistently across environments. In practice, containers themselves usually run on top of VMs in the cloud (e.g. AKS nodes are VMs that then run many containers), combining both layers of isolation.

LNX

CHAPTER 03 · LINUX BASICS

Linux Basics

The Linux filesystem, permissions, processes, users, and networking commands.

Linux is the operating system that powers the overwhelming majority of cloud servers, including most Azure Virtual Machines, all Kubernetes/AKS nodes, and nearly every open-source DevOps tool. A DevOps engineer who cannot navigate a Linux shell confidently cannot operate in the cloud — this section covers the commands, filesystem, permissions, and process/user/network management every cloud engineer uses daily.

💡 Layman Explanation

If Windows is like a car with an automatic transmission and a dashboard full of buttons, Linux (via the terminal) is like a manual transmission with direct access to the engine. It's less point-and-click, but it gives you precise, scriptable control — which is exactly what's needed to automate a fleet of hundreds of cloud servers identically.

3.1 The Linux Filesystem Hierarchy

```text
/  (root — top of everything, everything else lives under it)
├── /bin     → essential user command binaries (ls, cp, cat)
├── /sbin    → essential system binaries (reboot, iptables)
├── /etc     → system-wide configuration files (passwd, hosts, ssh)
├── /home    → personal home directories (/home/veera)
├── /root    → home directory of the root (superuser) account
├── /var     → variable data — logs (/var/log), spool, cache
├── /tmp     → temporary files, cleared on reboot
├── /opt     → optional third-party software packages
├── /usr     → user-installed programs and libraries
├── /proc    → virtual filesystem exposing live kernel/process info
├── /dev     → device files (disks, terminals) e.g. /dev/sda
└── /mnt     → mount point for temporarily mounted filesystems/disks
```

Fig 3: Linux Filesystem Hierarchy Standard (FHS)

3.2 All-Important Linux Commands

COMMAND	PURPOSE
`pwd`	Print current working directory
`ls -la`	List all files including hidden ones, with details (permissions, owner, size)
`cd <path>`	Change directory
`mkdir -p a/b/c`	Create nested directories in one command
`cp -r src/ dest/`	Copy files/directories recursively
`mv old new`	Move or rename a file/directory
`rm -rf dir/`	Remove a directory and contents forcibly (irreversible — use with care)
`cat file.txt`	Print file content to screen
`tail -f /var/log/syslog`	Follow a log file live, as new lines are written
`grep -r 'ERROR' /var/log/`	Search recursively for a pattern
`find / -name '*.log'`	Find files by name pattern across the filesystem
`chmod 755 script.sh`	Change file permissions
`chown user:group file`	Change file owner and group
`df -h`	Show disk space usage per mounted filesystem, human-readable
`du -sh folder/`	Show total size of a folder
`top` / `htop`	Live view of running processes and resource usage
`ps aux`	Snapshot list of all running processes
`kill -9 <pid>`	Force-terminate a process by its Process ID
`systemctl status nginx`	Check status of a systemd-managed service
`curl -I https://example.com`	Send an HTTP request and view response headers — useful for health checks

COMMAND	PURPOSE
`ssh user@host`	Connect to a remote Linux machine over SSH
`scp file.txt user@host:/path`	Securely copy a file to/from a remote machine
`tar -czvf archive.tar.gz folder/`	Compress a folder into a gzipped tarball
`history | grep git`	Search previously run shell commands

3.3 File Permissions

📝 Theory & Key Points

- Every file has 3 permission classes: Owner, Group, and Others, each with Read (r=4), Write (w=2), Execute (x=1).
- `chmod 755 file` means Owner=rwx(7), Group=r-x(5), Others=r-x(5) — the most common permission for scripts/executables.
- `chmod 644 file` is standard for regular files — Owner can read/write, everyone else can only read.
- Directories need the execute (x) bit to be 'entered' (cd into) — read (r) alone only lets you list names, not access contents.
- `sudo` temporarily elevates a command to run as root (superuser) — critical for installing packages or editing system files.

TEXT

```
-rwxr-xr-x  1 veera  devops  4096 Aug 12 10:15 deploy.sh
|└┘└┘└┘
| |  |  └── Others: r-x (read + execute, no write)
| |  └───── Group:  r-x (read + execute, no write)
| └──────── Owner:  rwx (read + write + execute)
└─────────── File type: '-' = regular file, 'd' = directory, 'l' = symlink
```

Reading 'ls -l' permission output

3.4 Process Management

BASH

```
# List all running processes with full details
ps aux

# Live, interactive resource monitor
top
htop            # nicer UI, install: sudo apt-get install htop -y

# Find the PID of a specific process
ps aux | grep nginx

# Gracefully stop a process (lets it clean up)
kill <PID>

# Force-kill an unresponsive process
kill -9 <PID>

# Run a command in the background, detached from the terminal
nohup python3 app.py &

# List background jobs of the current shell session
jobs
```

3.5 User Account Management

```bash
# Add a new user with a home directory
sudo useradd -m veera

# Set/change the password for a user
sudo passwd veera

# Add a user to the sudo (admin) group — Ubuntu/Debian
sudo usermod -aG sudo veera

# Delete a user and their home directory
sudo userdel -r veera

# Switch to another user
su - veera

# View currently logged-in users
who
whoami
```

3.6 Software Management

DISTRO FAMILY	PACKAGE MANAGER	EXAMPLE COMMANDS
Debian/Ubuntu	APT	`sudo apt-get update && sudo apt-get install nginx -y`
RHEL/CentOS/Amazon Linux	YUM / DNF	`sudo yum install nginx -y` / `sudo dnf install nginx -y`
Any distro (language-agnostic)	Snap	`sudo snap install docker`

3.7 Networking in Linux

COMMAND	PURPOSE
`ip a` / `ifconfig`	Show network interfaces and their IP addresses
`ping 8.8.8.8`	Test basic reachability to a host
`netstat -tulnp` / `ss -tulnp`	Show listening ports and which process owns them
`curl -v http://localhost:8080`	Verbose HTTP request — great for debugging app connectivity
`traceroute example.com`	Show the network hops a packet takes to reach a destination
`nslookup example.com` / `dig example.com`	Query DNS resolution for a domain
`ufw allow 22/tcp`	Allow a port through the host firewall (Ubuntu's uncomplicated firewall)

🎯 Scenario-Based Interview Questions

Q: Your Azure Linux VM's disk is filling up and an application is crashing due to 'no space left on device'. Walk through how you diagnose and fix it. First run `df -h` to see which mounted filesystem is full (e.g. /var at 100%). Then run `du -sh /var/* | sort -rh | head -10` to find the largest directories — very often /var/log has huge, unrotated log files. Use `find /var/log -name '*.log' -size +100M` to pinpoint offenders, then either truncate (`truncate -s 0 file.log`) or remove old rotated logs. For a lasting fix, configure logrotate, and if the underlying issue is a genuinely undersized disk, resize the Azure Managed Disk and extend the filesystem with `growpart` and `resize2fs` (or `xfs_growfs` for XFS).

Q: A new team member accidentally ran a script with the wrong permissions and it silently failed. How would you check and fix the permission issue on a deployment script? Run `ls -l deploy.sh` to inspect current permissions — most likely the execute bit is missing (e.g. `-rw-r--r--`). Fix it with `chmod +x deploy.sh` (or explicitly `chmod 755 deploy.sh`), then verify ownership with `ls -l` again to confirm the correct owner:group can execute it. If the script needs to run as a service account rather than the current user, also set ownership via `chown svc-user:svc-group deploy.sh` . As a long-term practice, add proper permission bits into the deployment pipeline (e.g. Git preserves the executable bit) so this can't recur.

VM

CHAPTER 04 · COMPUTE SERVICES

Azure Virtual Machines — Overview & Instance Launch

Azure Virtual Machines — images, sizes, and launching your first instance.

An Azure Virtual Machine (VM) is Azure's core IaaS compute offering — an on-demand, scalable computing resource that behaves exactly like a physical server, giving you full control over the OS, so you can install any software just as you would on your own hardware. Azure VMs are the direct equivalent of AWS EC2 instances.

💡 Layman Explanation

An Azure VM is like renting a fully furnished apartment (a slice of a physical server) instead of building your own house. You choose the size (small studio vs large penthouse = VM size), the neighbourhood (region), and you get the keys (admin/root access) instantly — no construction time.

4.1 VM Overview — Core Concepts

📝 Theory & Key Points

- A VM is defined by: **Image** (the OS template — Windows Server, Ubuntu, RHEL), **Size/SKU** (vCPU + RAM + disk throughput class), **Disk** (OS disk + optional data disks), **Network Interface** (private/public IP, NSG), and **Region/Availability Zone**.
- Azure VM series map to workload types: **B-series** (burstable, cheap, dev/test), **D-series** (general purpose), **E-series** (memory-optimised, databases), **F-series** (compute-optimised), **N-series** (GPU, AI/ML/rendering).
- VMs can be deployed from the Azure Marketplace (pre-built images: Windows, Ubuntu, RHEL, SQL Server pre-installed) or from a custom image you build yourself.
- Every VM needs an **NIC (Network Interface Card)** attached to a VNET subnet, and typically an NSG to control inbound/outbound traffic.

CONCEPT	AZURE TERM	AWS EQUIVALENT
Virtual server	Virtual Machine (VM)	EC2 Instance
OS template	VM Image / Azure Marketplace Image	AMI (Amazon Machine Image)
Size class	VM Size (e.g. Standard_D2s_v5)	Instance Type (e.g. t3.medium)
SSH/RDP key pair	SSH public key / admin password	Key Pair (.pem)
Firewall	Network Security Group (NSG)	Security Group

4.2 Launching an Instance — Linux VM

BASH

```
# Login to Azure
az login

# Create a resource group (logical container for related resources)
az group create --name rg-devops-lab --location centralindia

# Create an Ubuntu VM with SSH key auth
az vm create \
  --resource-group rg-devops-lab \
  --name vm-web01 \
  --image Ubuntu2204 \
  --size Standard_B2s \
  --admin-username veera \
  --generate-ssh-keys \
  --public-ip-sku Standard

# Open port 22 (SSH) and 80 (HTTP) for inbound traffic
az vm open-port --resource-group rg-devops-lab --name vm-web01 --port 22
az vm open-port --resource-group rg-devops-lab --name vm-web01 --port 80

# Get the public IP to connect
az vm show -d --resource-group rg-devops-lab --name vm-web01 --query publicIps -o tsv

# SSH into the VM
ssh veera@<public-ip>
```

4.3 Launching an Instance — Windows VM

BASH

```
# Create a Windows Server VM with admin username/password
az vm create \
  --resource-group rg-devops-lab \
  --name vm-win01 \
  --image Win2022Datacenter \
  --size Standard_D2s_v5 \
  --admin-username azureadmin \
  --admin-password "P@ssw0rd12345!"

# Open RDP port 3389
az vm open-port --resource-group rg-devops-lab --name vm-win01 --port 3389

# Then connect via Remote Desktop (mstsc.exe on Windows, or Microsoft Remote Desktop
on Mac)
mstsc /v:<public-ip>
```

⚠️ Important Warning

- Never embed a plaintext admin password inside a script committed to Git — use Azure Key Vault references or prompt interactively.
- Always restrict RDP (3389) and SSH (22) to your own IP range in the NSG rather than 'Any' — internet-wide open management ports are one of the most common breach vectors.

🎯 Scenario-Based Interview Questions

Q: Your manager wants a quick dev VM for a 2-hour testing session and wants to minimize cost. Which VM series would you pick and why? A B-series (burstable) VM such as Standard_B2s. B-series VMs accumulate CPU credits during idle periods and burst above their baseline when needed, and are priced significantly lower than D-series general-purpose VMs for the same vCPU/RAM count. For a short-lived, low, bursty testing workload this is the most cost-efficient choice — combined with deallocating (not just stopping) the VM immediately after the session, since Azure only stops billing compute once a VM is fully deallocated (stopped VMs that are merely 'stopped' but not deallocated still incur compute charges).

VMOP

CHAPTER 05 · COMPUTE SERVICES

Azure VM Instance Management

VM lifecycle management — SSH keys, NSGs, disks, snapshots, and metadata.

Beyond simply launching a VM, real-world operations require managing its lifecycle: choosing the right size for the workload, handling SSH key access securely, controlling network traffic with firewalls, managing disk storage and backups, and passing configuration data into the VM at boot time.

5.1 Instance Types (VM Size Families)

SERIES	OPTIMISED FOR	EXAMPLE SIZES
A-series	Entry-level, dev/test	A1_v2, A2_v2
B-series	Burstable, cost-sensitive workloads	B1s, B2s, B4ms
D-series (v5)	General purpose (balanced CPU:RAM)	D2s_v5, D4s_v5, D8s_v5
E-series (v5)	Memory-optimised (databases, caches)	E4s_v5, E8s_v5
F-series (v2)	Compute-optimised (batch, gaming, web servers)	F2s_v2, F4s_v2
N-series	GPU-accelerated (ML, rendering)	NC6s_v3, ND40rs_v2
L-series	Storage-optimised, high disk throughput	L8s_v3

5.2 Key Pair (SSH) Management

📝 Theory & Key Points

- Azure Linux VMs authenticate via SSH public/private key pairs by default (more secure than passwords) — the public key is injected into the VM's `~/.ssh/authorized_keys` at creation.
- The private key must be downloaded and stored securely — Azure never stores your private key.
- Keys can also be stored and managed centrally in Azure Key Vault for enterprise key rotation and access auditing.
- For Windows VMs, authentication is typically username/password, or certificate-based for domain-joined machines.

BASH

```
# Generate a new SSH key pair locally
ssh-keygen -t rsa -b 4096 -f ~/.ssh/azure_devops_key -C "veera@devops"

# Create a VM using your existing public key
az vm create --resource-group rg-devops-lab --name vm-app01 \
  --image Ubuntu2204 --admin-username veera \
  --ssh-key-values ~/.ssh/azure_devops_key.pub

# Add an additional SSH key to an existing running VM (via VM Access extension)
az vm user update --resource-group rg-devops-lab --name vm-app01 \
  --username veera --ssh-key-value ~/.ssh/second_key.pub
```

5.3 Security Groups & Firewalls (NSG)

📝 Theory & Key Points

- A **Network Security Group (NSG)** is a stateful firewall — a set of allow/deny rules evaluated by priority (lower number = higher priority) that control inbound and outbound traffic at the subnet or NIC level.
- NSGs are stateful: if inbound traffic is allowed, the corresponding outbound response is automatically allowed — you don't need a matching outbound rule.
- Azure adds default rules at the bottom of every NSG (e.g. AllowVNetInBound, DenyAllInBound) that cannot be deleted, only overridden by higher-priority custom rules.
- Best practice: attach NSGs at the subnet level for consistent policy across all VMs in that subnet, and use Application Security Groups (ASGs) to group VMs logically (e.g. 'WebTier') instead of hardcoding IPs.

BASH

```
# Create an NSG
az network nsg create --resource-group rg-devops-lab --name nsg-web

# Allow inbound HTTPS only from a specific IP range
az network nsg rule create --resource-group rg-devops-lab --nsg-name nsg-web \
  --name Allow-HTTPS --priority 100 --direction Inbound --access Allow \
  --protocol Tcp --destination-port-ranges 443 --source-address-prefixes
203.0.113.0/24

# Deny all other inbound traffic explicitly (lower priority number = evaluated first)
az network nsg rule create --resource-group rg-devops-lab --nsg-name nsg-web \
  --name Deny-All-Inbound --priority 4096 --direction Inbound --access Deny --protocol
'*'
```

5.4 Storage and Snapshots

📝 Theory & Key Points

- Every VM has an **OS disk** (boot volume) and can attach additional **Data disks** (Managed Disks) for application data, independent of the VM lifecycle.
- A **Snapshot** is a read-only, point-in-time full copy of a managed disk — used for backup, cloning environments, or forensic investigation.
- Disk performance tiers: Standard HDD (cheapest, backups/archives), Standard SSD (balanced, dev/test), Premium SSD (production, low-latency), Ultra Disk (highest IOPS, SAP HANA/large databases).

BASH

```
# Attach a new 128GB data disk to a running VM
az vm disk attach --resource-group rg-devops-lab --vm-name vm-app01 \
  --name disk-appdata --size-gb 128 --new --sku Premium_LRS

# Take a snapshot of the OS disk for backup
az snapshot create --resource-group rg-devops-lab --name snap-vmapp01-osdisk \
  --source vm-app01_OsDisk_1

# Create a new disk from that snapshot (e.g. to clone into another VM)
az disk create --resource-group rg-devops-lab --name disk-restored \
  --source snap-vmapp01-osdisk
```

5.5 User Data and Metadata

📝 Theory & Key Points

- **Custom Data / User Data** is a script (cloud-init for Linux, PowerShell for Windows) passed at VM creation time to automatically bootstrap software — e.g. installing NGINX and starting it on first boot, without manual login.
- **Instance Metadata Service (IMDS)** is a REST endpoint reachable only from inside the VM at `http://169.254.169.254/metadata/instance` that lets an application query its own VM ID, region, tags, and network config at runtime.
- IMDS is also how a VM retrieves a Managed Identity access token to authenticate to other Azure services without embedding any secret credentials.

BASH

```
# cloud-init script to auto-install and start NGINX at boot
cat > cloud-init.txt <<'EOF'
#cloud-config
package_upgrade: true
packages:
  - nginx
runcmd:
  - systemctl enable nginx
  - systemctl start nginx
EOF

az vm create --resource-group rg-devops-lab --name vm-web02 \
  --image Ubuntu2204 --admin-username veera --generate-ssh-keys \
  --custom-data cloud-init.txt

# Query instance metadata from INSIDE the VM
curl -H "Metadata:true" \
  "http://169.254.169.254/metadata/instance?api-version=2021-02-01" | jq
```

🎯 Scenario-Based Interview Questions

Q: Your web tier VMs need outbound-only internet access for OS updates, but must never accept unsolicited inbound connections from the internet, only from an internal Application Gateway subnet. How do you configure this? Create an NSG on the web-tier subnet with a single high-priority inbound Allow rule scoped to the Application Gateway subnet's address range on the app port (e.g. 443), an inbound Deny-All rule below it, and leave the default AllowVNetOutBound/AllowInternetOutBound rules intact so the VMs can still reach Azure/OS update endpoints outbound. Because NSGs are stateful, response traffic to those outbound requests is automatically permitted without needing an explicit inbound rule — inbound is only opened for the Application Gateway's specific subnet.

LB

CHAPTER 06 · COMPUTE SERVICES

Azure Load Balancer

Distributing traffic — Load Balancer, Application Gateway, Traffic Manager, Front Door.

A Load Balancer distributes incoming network traffic across multiple backend VM instances so that no single instance is overwhelmed, and so that if one instance fails, traffic is automatically routed to the healthy ones. It is a foundational building block for any highly available, horizontally-scaled architecture.

💡 Layman Explanation

A load balancer is like the host at a busy restaurant who seats new customers at whichever table is free, instead of letting everyone crowd around one table while others sit empty. If a table (server) becomes unavailable (a VM crashes), the host simply stops sending customers there until it's back.

6.1 Types of Azure Load Balancers

LOAD BALANCER TYPE	OSI LAYER	USE CASE
Azure Load Balancer	Layer 4 (TCP/UDP)	High-performance, low-latency load balancing for any TCP/UDP traffic — VM scale sets, generic backend pools.
Azure Application Gateway	Layer 7 (HTTP/HTTPS)	Web traffic needing URL-path routing, SSL termination, cookie-based session affinity, and integrated Web Application Firewall (WAF).
Azure Traffic Manager	DNS-based (Global)	Routes users to the closest/healthiest Azure region across the globe — not a per-request balancer but a DNS-level router.
Azure Front Door	Layer 7 (Global, CDN-integrated)	Global HTTP(S) load balancing with edge caching, WAF, and SSL offload, similar to AWS CloudFront + ALB combined.

6.2 Public vs Internal Load Balancer

📝 Theory & Key Points

- **Public Load Balancer**: has a public IP, exposed to the internet — used for internet-facing applications (e.g. a public website).
- **Internal (Private) Load Balancer**: only has a private IP inside a VNET — used for internal tiers, e.g. balancing traffic from a web tier to an internal API/database tier that should never be reachable from the internet.
- A **Backend Pool** defines which VMs/VMSS instances receive traffic.
- A **Health Probe** continuously checks each backend instance (e.g. HTTP GET /healthz every 15s); instances that fail the probe are automatically removed from rotation until healthy again.
- **Load Balancing Rules** map a frontend IP+port to a backend pool+port, with a chosen distribution mode (5-tuple hash by default).

BASH

```
# Create a public Standard Load Balancer
az network lb create --resource-group rg-devops-lab --name lb-web \
  --sku Standard --public-ip-address pip-lb-web \
  --frontend-ip-name feConfig --backend-pool-name webBackendPool

# Create a health probe checking /healthz over HTTP
az network lb probe create --resource-group rg-devops-lab --lb-name lb-web \
  --name healthProbe --protocol Http --port 80 --path /healthz

# Create the load balancing rule: frontend port 80 -> backend pool port 80
az network lb rule create --resource-group rg-devops-lab --lb-name lb-web \
  --name httpRule --protocol Tcp --frontend-port 80 --backend-port 80 \
  --frontend-ip-name feConfig --backend-pool-name webBackendPool --probe-name
healthProbe
```

🎯 Scenario-Based Interview Questions

Q: Your 3-tier app has a public web tier and an internal API tier that the database team says must never be internet-reachable. How do you load-balance traffic to the API tier while enforcing that isolation? Deploy an Internal Load Balancer (ILB) with only a private frontend IP inside the API tier's subnet — it never gets a public IP, so it is structurally unreachable from the internet regardless of NSG configuration. The web tier calls the ILB's private IP/DNS name for API requests. Combine this with an NSG on the API subnet that only allows inbound traffic from the web tier's subnet range, giving defense in depth (network isolation from the ILB design + explicit NSG rule) rather than relying on a single control.

MON

CHAPTER 07 · COMPUTE SERVICES

Billing and Monitoring

Cost management and Azure Monitor — budgets, metrics, alerts, KQL.

Cloud cost visibility and operational monitoring are two sides of the same coin: monitoring tells you if your systems are healthy, while cost management tells you if you're paying a sensible amount to keep them that way. Azure provides Azure Monitor for observability and Azure Cost Management + Billing for financial governance.

7.1 Azure Cost Management & Billing

📝 Theory & Key Points

- **Azure Cost Management** provides cost analysis dashboards, broken down by resource group, service, tag, or subscription, so teams can see exactly where spend is going.
- **Budgets** can be set per resource group or subscription with automated email/action-group alerts when spend crosses a threshold (e.g. 80% of monthly budget).
- **Tags** (key-value pairs like `env=prod`, `team=devops`) attached to resources are essential for cost allocation/chargeback across teams — untagged resources are the #1 cause of 'mystery spend'.
- **Azure Advisor** proactively recommends cost-saving actions — e.g. flagging an underutilised VM that could be resized down, or an unattached disk that's still being billed.
- Cost export can be scheduled to a Storage Account as CSV for further analysis in Power BI or Excel.

7.2 Azure Monitor & Alarms

📝 Theory & Key Points

- **Azure Monitor** is the umbrella observability platform collecting metrics (numeric time-series like CPU %) and logs (Activity Logs, Diagnostic Logs, Application Insights traces) across every Azure resource.
- **Metrics** are near-real-time numeric values (e.g. Percentage CPU, Network In/Out) queryable and chartable with low latency.
- **Log Analytics Workspace** stores logs and lets you query them with **KQL (Kusto Query Language)** — Azure's equivalent of CloudWatch Logs Insights.
- **Alerts (Alarms)** are rules defined on a metric or log query that trigger an **Action Group** (email, SMS, webhook, Azure Function, auto-scale action) when a threshold is breached.
- **Application Insights** is Azure Monitor's Application Performance Monitoring (APM) feature — auto-instruments code to track request latency, dependency calls, exceptions, and distributed traces.

BASH

```
# Create a budget with an alert at 80% of $500/month, emailing the team
az consumption budget create --budget-name monthly-devops-budget \
  --amount 500 --time-grain Monthly \
  --category Cost --start-date 2026-08-01 --end-date 2027-08-01

# Create a metric alert: fire if average CPU > 85% for 5 minutes on a VM
az monitor metrics alert create --resource-group rg-devops-lab \
  --name high-cpu-alert --scopes /subscriptions/<sub-id>/resourceGroups/rg-devops-
lab/providers/Microsoft.Compute/virtualMachines/vm-web01 \
  --condition "avg Percentage CPU > 85" --window-size 5m --evaluation-frequency 1m \
  --action myActionGroup
```

```kql
// KQL: find the top 10 slowest requests in the last 24 hours (Application Insights)
requests
| where timestamp > ago(24h)
| top 10 by duration desc
| project timestamp, name, duration, resultCode
```

🎯 Scenario-Based Interview Questions

Q: Finance reports the Azure bill jumped 40% this month with no corresponding traffic increase. How do you investigate? Start in Azure Cost Management → Cost Analysis, grouped by Resource Group and then by Service, comparing this month vs last. Look for anomalies: a common cause is orphaned resources — unattached Managed Disks or Public IPs left after a VM was deleted, forgotten dev/test VMs left running 24/7, or a VM Scale Set autoscale rule that scaled out and never scaled back in due to a misconfigured cooldown/threshold. Cross-check Azure Advisor's cost recommendations for underutilised or idle resources, and review the Activity Log for who created new resources and when. Once identified, deallocate/delete the waste and enforce a tagging + budget-alert policy so it's caught within days next time, not a full billing cycle.

SCALE

CHAPTER 08 · COMPUTE SERVICES

Azure Auto Scaling

Virtual Machine Scale Sets and reactive/scheduled/predictive autoscaling.

Auto Scaling automatically adjusts the number of running compute instances in response to real-time demand, ensuring an application has enough capacity to handle load spikes while avoiding the cost of running excess capacity during quiet periods. In Azure, this is implemented primarily through Virtual Machine Scale Sets (VMSS).

8.1 Virtual Machine Scale Sets (VMSS)

📝 Theory & Key Points

- A **VM Scale Set** is a group of identical, load-balanced VMs deployed from a single configuration (image + size + extensions), managed as one logical unit — Azure's equivalent of an AWS Auto Scaling Group.
- VMSS supports up to 1,000 VM instances (600 with a custom image) and integrates natively with Azure Load Balancer or Application Gateway.
- VMSS instances can be spread across multiple **Availability Zones** automatically for higher resilience against a datacenter-level failure.

8.2 Scaling Types and Policies

SCALING TYPE	TRIGGER	EXAMPLE
Manual Scaling	Human sets a fixed instance count	Set instance count to 5 before a planned marketing campaign.
Metric-Based (Reactive) Autoscale	A metric crosses a threshold (CPU, memory, queue length, custom Application Insights metric)	Scale out by 2 instances when average CPU > 75% for 10 minutes.
Schedule-Based Autoscale	A time-of-day/day-of-week schedule	Scale to 10 instances every weekday 9 AM, scale to 2 instances at 8 PM.
Predictive Autoscale	Machine-learning forecast of historical demand patterns, pre-scaling ahead of expected load	Scale up ahead of a recurring Friday evening traffic spike, before CPU actually rises.

8.3 Azure Auto-Scaling — Configuration

BASH

```
# Create a VM Scale Set behind a load balancer, 2 initial instances
az vmss create --resource-group rg-devops-lab --name vmss-web \
  --image Ubuntu2204 --instance-count 2 --vm-sku Standard_D2s_v5 \
  --admin-username veera --generate-ssh-keys \
  --load-balancer lb-web --upgrade-policy-mode Automatic

# Attach an autoscale profile: scale out on high CPU, scale in on low CPU
az monitor autoscale create --resource-group rg-devops-lab \
  --resource vmss-web --resource-type Microsoft.Compute/virtualMachineScaleSets \
  --name autoscale-web --min-count 2 --max-count 10 --count 2

az monitor autoscale rule create --resource-group rg-devops-lab \
  --autoscale-name autoscale-web --scale out 2 \
  --condition "Percentage CPU > 75 avg 10m"

az monitor autoscale rule create --resource-group rg-devops-lab \
  --autoscale-name autoscale-web --scale in 1 \
  --condition "Percentage CPU < 25 avg 10m"
```

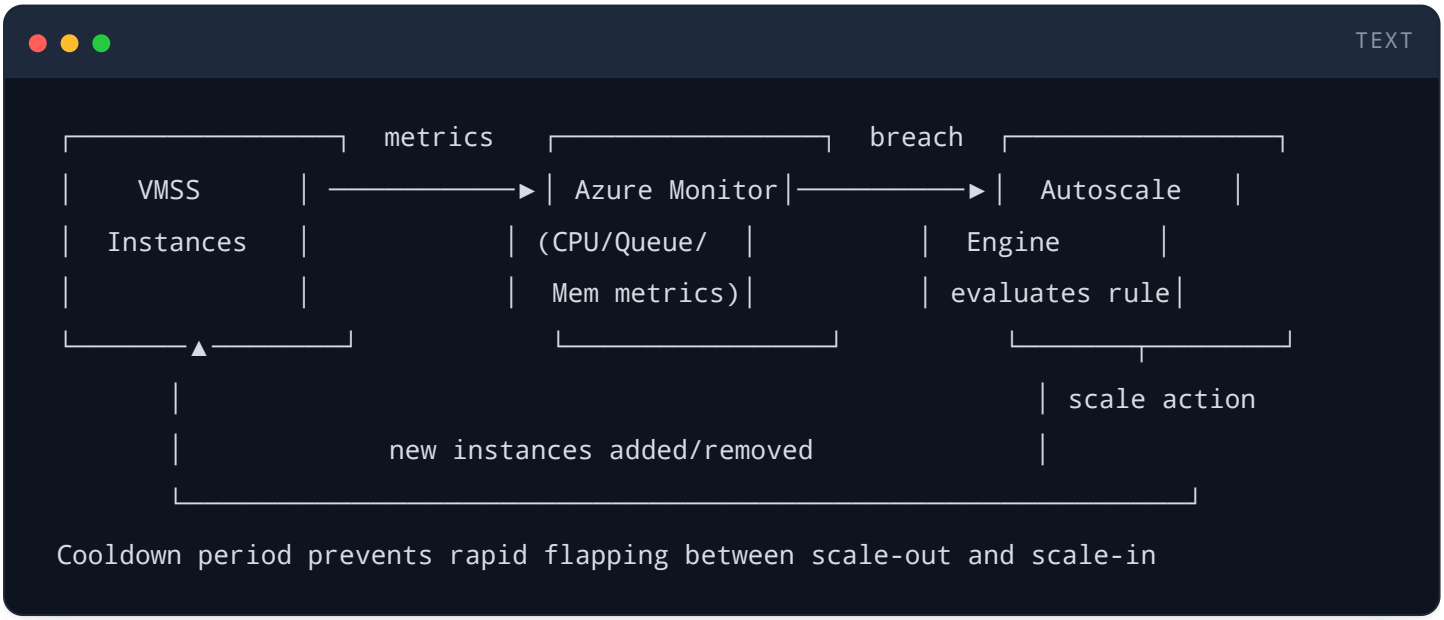


Fig 4: Reactive Autoscale Feedback Loop

⚠️ Important Warning

- Always set a sensible cooldown period between scale actions — without it, a fluctuating metric can cause 'flapping' (rapid scale out/in), which is both costly and destabilising.
- Scaling in (removing instances) should be tied to graceful connection draining so in-flight requests aren't dropped mid-response.

🎯 Scenario-Based Interview Questions

Q: An e-commerce site knows it gets a predictable traffic spike every day at 6 PM when people shop after work, but reactive CPU-based autoscaling always lags 5-10 minutes behind the spike, causing brief slowness. How do you fix this? Combine schedule-based autoscaling with the existing reactive rule instead of relying on CPU alone. Add a scheduled profile that pre-scales the VMSS to a higher minimum instance count (e.g. from 2 to 6) at 5:45 PM, ahead of the known 6 PM spike, so capacity is already in place before demand arrives rather than reacting after CPU crosses the threshold. Keep the reactive CPU rule active as a safety net for unpredictable spikes on top of the scheduled baseline, and scale back down to the lower baseline late at night via a second scheduled profile.

DISK

CHAPTER 09 · STORAGE SERVICES

Storage Services — Managed Disks & Azure Files

Managed Disks and Azure Files — block storage vs shared file storage.

Compute needs persistent storage attached to it. Azure offers two distinct storage services for this: Managed Disks (block storage attached to a single VM, equivalent to AWS EBS) and Azure Files (network file shares accessible by many VMs simultaneously, equivalent to AWS EFS).

9.1 Azure Managed Disks (EBS Equivalent)

📝 Theory & Key Points

- A **Managed Disk** is a virtual hard disk (VHD) fully managed by Azure — you specify size and performance tier, and Azure handles the underlying storage hardware, replication, and availability automatically (unmanaged disks, which required manually managing storage accounts, are legacy/deprecated).
- A Managed Disk attaches to exactly **one VM at a time** (except Ultra Disks/Premium SSD v2 with shared disk enabled for clustered workloads) — block storage, not shared file storage.
- Disks can be resized live in many cases, detached and reattached to a different VM, and snapshotted independently of the VM's lifecycle.
- Data persists independently of the VM — deleting a VM does not delete its disks unless explicitly configured to do so.

DISK TIER	PERFORMANCE	TYPICAL USE
Standard HDD	Lowest cost, higher latency	Backup, infrequently accessed data
Standard SSD	Balanced cost/performance	Dev/test, low-IOPS web servers
Premium SSD	Low latency, consistent high IOPS	Production databases, business-critical VMs
Premium SSD v2	Independently tunable IOPS/throughput without resizing capacity	High-performance production workloads needing fine cost control
Ultra Disk	Sub-millisecond latency, highest IOPS/throughput, can be shared	SAP HANA, large transactional databases

9.2 Azure Files (EFS Equivalent)

📝 Theory & Key Points

- **Azure Files** provides fully managed file shares accessible concurrently via the standard **SMB** and **NFS** protocols — multiple VMs (or even on-prem servers via VPN/ExpressRoute) can mount the same share simultaneously.
- Unlike a Managed Disk, Azure Files is shared network storage, making it ideal for shared configuration, shared web content across a scaled-out web tier, or lift-and-shift of legacy apps expecting a network drive.
- Supports Azure File Sync to cache frequently-used files on an on-prem Windows Server while keeping the full data set in the cloud.
- Storage tiers: Premium (SSD-backed, low latency), Transaction Optimized, Hot, and Cool for cost-tiering less-frequently accessed shares.

ASPECT	MANAGED DISK (BLOCK STORAGE)	AZURE FILES (NETWORK FILE SHARE)
AWS Equivalent	EBS	EFS
Attach model	One VM at a time	Many clients concurrently
Protocol	Block device (mounted as a raw disk)	SMB / NFS
Typical use	OS disk, database data files	Shared config, shared uploads, lift-and-shift file servers

BASH

```
# Create a 256GB Premium SSD managed disk
az disk create --resource-group rg-devops-lab --name disk-db-data \
  --size-gb 256 --sku Premium_LRS

# Attach it to an existing VM
az vm disk attach --resource-group rg-devops-lab --vm-name vm-db01 \
  --name disk-db-data

# Create a Storage Account + a 100GB Azure Files share
az storage account create --resource-group rg-devops-lab --name stdevopslab001 \
  --sku Standard_LRS --kind StorageV2

az storage share-rm create --resource-group rg-devops-lab \
  --storage-account stdevopslab001 --name sharedconfig --quota 100

# Mount the file share on a Linux VM
sudo mount -t cifs //stdevopslab001.file.core.windows.net/sharedconfig /mnt/config \
  -o vers=3.0,username=stdevopslab001,password=<storage-
key>,dir_mode=0777,file_mode=0777
```

🎯 Scenario-Based Interview Questions

Q: You're migrating an on-prem 3-node web farm that all read/write to a shared 'uploads' folder on a Windows file server. What's the right Azure storage choice, and why not a Managed Disk?
Azure Files is the correct choice, mounted via SMB and attached to all three web VMs simultaneously, exactly replicating the shared-folder behaviour they rely on today. A Managed Disk cannot serve this use case because it attaches to only one VM at a time — using it would force all uploads through a single VM, breaking the multi-node architecture. Azure Files also supports Azure File Sync if they want to keep a local on-prem cache during a phased migration.

VNET

CHAPTER 10 · NETWORKING

Networking — Virtual Network (VNET)

Virtual Network fundamentals — subnets, CIDR, public vs private IPs.

An Azure Virtual Network (VNET) is the fundamental building block of your private network in the cloud — a logically isolated section of Azure where you launch VMs and other resources, with full control over IP address ranges, subnets, route tables, and gateways. It is the direct equivalent of an AWS VPC.

💡 Layman Explanation

A VNET is your own private office building inside a shared skyscraper (the Azure region). You decide the floor plan (subnets), who can enter each room (NSGs), and how rooms connect to each other and to the outside world.

10.1 Networking Basics

📝 Theory & Key Points

- A VNET is scoped to a single Azure **region** and a single **subscription**, defined by an address space in CIDR notation (e.g. 10.0.0.0/16).
- A VNET is divided into one or more **Subnets**, each a smaller CIDR block carved from the VNET's range — resources (VMs, App Service integration, AKS nodes) are deployed into a specific subnet.
- Azure reserves the first 4 and the last IP address in every subnet for internal use (network address, default gateway, two DNS reserved addresses, broadcast) — so a /24 subnet (256 addresses) only has 251 usable addresses.
- VNETs are private by default — no traffic flows in or out until you explicitly configure NSGs, public IPs, gateways, or peering.

10.2 Public IPs vs Private IPs

TYPE	REACHABLE FROM	TYPICAL USE
Private IP	Only within the VNET (or peered/VPN-connected networks)	Internal communication between app tiers, databases
Public IP	The internet	Load balancer frontends, jump boxes/bastion, public-facing endpoints

10.3 CIDR Range Basics

CIDR	SUBNET MASK	TOTAL ADDRESSES	USABLE IN AZURE SUBNET
/16	255.255.0.0	65,536	65,531
/24	255.255.255.0	256	251
/28	255.255.255.240	16	11
/29	255.255.255.248	8	3 (minimum practical Azure subnet size)

10.4 Subnet Calculation Example

TEXT

```
VNET: 10.0.0.0/16   (65,536 total addresses: 10.0.0.0 - 10.0.255.255)

┌──────────────────────────────────────────────────────────────────┐
│  Subnet: snet-web      10.0.1.0/24   (251 usable IPs)    │
├──────────────────────────────────────────────────────────────────┤
│  Subnet: snet-app      10.0.2.0/24   (251 usable IPs)    │
├──────────────────────────────────────────────────────────────────┤
│  Subnet: snet-data     10.0.3.0/24   (251 usable IPs)    │
├──────────────────────────────────────────────────────────────────┤
│  Subnet: snet-gateway  10.0.255.0/27 (dedicated GatewaySubnet)│
└──────────────────────────────────────────────────────────────────┘

Remaining 10.0.4.0/22 - 10.0.254.0/24 space kept free for future growth
```

Fig 5: Carving Subnets from a VNET's Address Space

10.5 Creating Custom VNETs and Subnets

BASH

```
# Create a VNET with an initial subnet
az network vnet create --resource-group rg-devops-lab --name vnet-prod \
  --address-prefix 10.0.0.0/16 \
  --subnet-name snet-web --subnet-prefix 10.0.1.0/24

# Add additional subnets to the same VNET
az network vnet subnet create --resource-group rg-devops-lab --vnet-name vnet-prod \
  --name snet-app --address-prefix 10.0.2.0/24

az network vnet subnet create --resource-group rg-devops-lab --vnet-name vnet-prod \
  --name snet-data --address-prefix 10.0.3.0/24

# List all subnets in a VNET
az network vnet subnet list --resource-group rg-devops-lab --vnet-name vnet-prod -o
table
```

🎯 Scenario-Based Interview Questions

Q: You are designing a VNET for a 3-tier app (web, app, data) that must scale to hundreds of instances per tier over the next 3 years, plus a future AKS cluster. How do you size the address space? Start with a generously sized VNET such as 10.0.0.0/16 (65,536 addresses) rather than a tight /24, since VNET address ranges are painful to expand later if they overlap with peered networks. Allocate a /22 or larger to each tier's subnet (1,024+ addresses) to comfortably absorb horizontal scaling and, critically, reserve an extra-large subnet for AKS — Azure CNI-networked AKS clusters consume one IP per pod by default, so even a modest cluster can need thousands of addresses. Leave large unallocated ranges between subnets for future tiers, and document the plan so it doesn't collide with any on-prem or peered VNET ranges during a future VPN/ExpressRoute connection.

VNETC

CHAPTER 11 · NETWORKING

Networking — VNET Controls

NSGs as ACLs, route tables, VPN/ExpressRoute gateways, hub-and-spoke.

Once a VNET and its subnets exist, two mechanisms control how traffic actually flows: Network Security Groups (filtering) and Route Tables + Gateways (directing traffic to its destination, including out to the internet or back to on-premises).

11.1 Network Security Groups — ACL Equivalent

📝 Theory & Key Points

- An NSG functions like a stateful Access Control List (ACL) / Security Group — a prioritized list of allow/deny rules for inbound and outbound traffic, matched on source/destination IP, port, and protocol.
- NSGs can attach to a **subnet** (applies to every resource in it) or directly to a **NIC** (applies to just that one VM) — both can be combined, with the most restrictive combined result winning.
- Rule evaluation is by priority number (100-4096, lower = evaluated first); once a rule matches, evaluation stops for that packet.
- Unlike a traditional stateless network ACL, Azure NSGs are stateful for both allow and deny at the flow level — but note Azure also has a separate concept: subnet-level NSGs feel like a 'network ACL', while VMSS/NIC-level NSGs feel like per-instance 'security groups', mirroring the two AWS concepts in one Azure construct.

11.2 Route Tables & Gateways

📝 Theory & Key Points

- By default, Azure automatically creates **System Routes** so traffic between subnets in the same VNET, and outbound to the internet, works without configuration.
- A **Route Table (UDR – User Defined Route)** lets you override this default behaviour — e.g. force all outbound traffic from a subnet through a central Network Virtual Appliance (firewall) instead of going directly to the internet.
- A **VPN Gateway** connects your VNET to an on-premises network (or another cloud) over an encrypted IPsec tunnel across the public internet.
- An **ExpressRoute Gateway** connects on-prem to Azure over a private, dedicated circuit (not traversing the public internet) — used when consistent low latency and higher bandwidth are required (equivalent to AWS Direct Connect).
- A **NAT Gateway** gives an entire subnet a single, consistent outbound public IP for internet access, without exposing individual VMs with their own public IPs.

TEXT

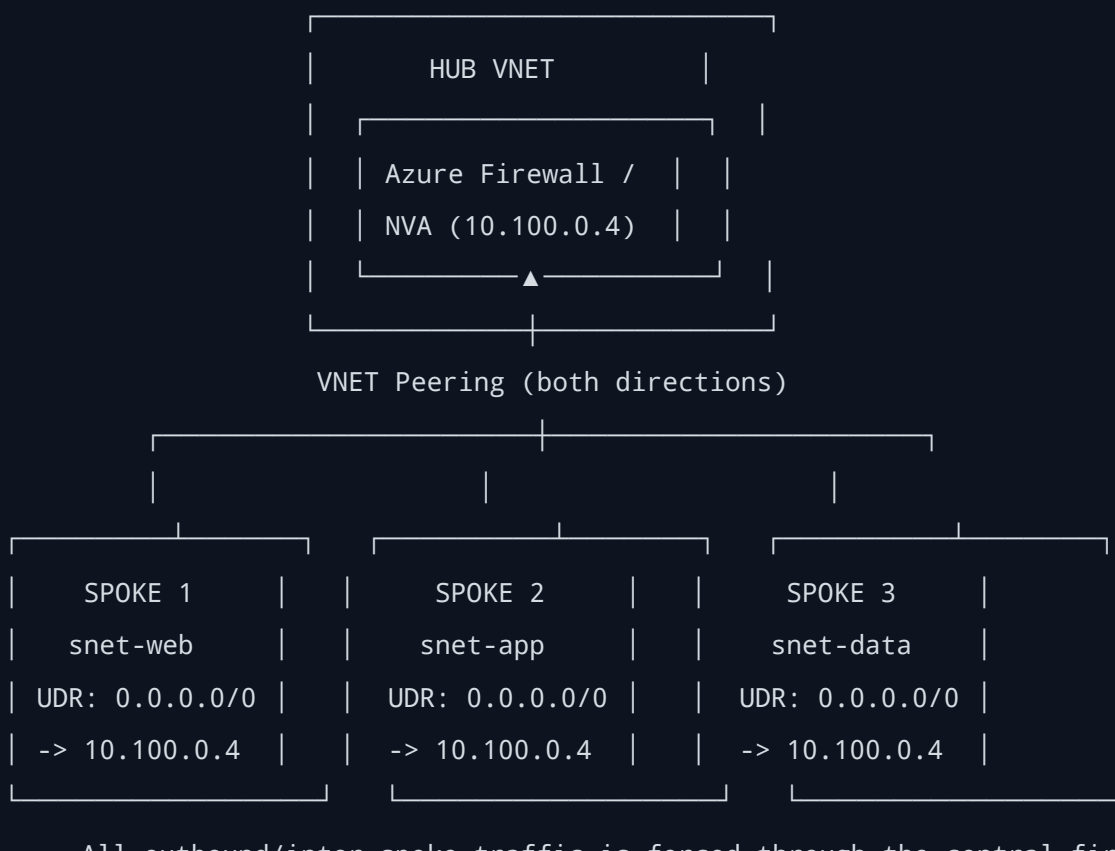


Fig 6: Hub-and-Spoke Network with UDR Forcing Traffic Through a Firewall

BASH

```
# Create a route table that forces traffic through a central firewall NVA
az network route-table create --resource-group rg-devops-lab --name rt-spoke-web

az network route-table route create --resource-group rg-devops-lab \
  --route-table-name rt-spoke-web --name default-via-firewall \
  --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance \
  --next-hop-ip-address 10.100.0.4

# Associate the route table with the web subnet
az network vnet subnet update --resource-group rg-devops-lab \
  --vnet-name vnet-prod --name snet-web --route-table rt-spoke-web
```

🎯 Scenario-Based Interview Questions

Q: Security wants all outbound internet traffic from every VNET to be inspected by a central firewall before leaving Azure, rather than each subnet reaching the internet directly. How do you architect this at the network level? Adopt a hub-and-spoke topology: a central Hub VNET hosts Azure Firewall (or a third-party NVA), and each application's VNET becomes a Spoke, VNET-peered to the hub. On every spoke subnet, apply a User Defined Route (UDR) with a default route (0.0.0.0/0) whose next hop is the firewall's private IP in the hub, overriding Azure's default direct-to-internet system route. This forces all outbound traffic to traverse the firewall for inspection/logging before egress, while VNET peering keeps latency low since traffic never leaves Azure's backbone network to reach the hub.

BLOB

CHAPTER 12 · OBJECT STORAGE (BLOB)

Object Storage — Azure Blob Storage

Azure Blob Storage fundamentals — containers, tiers, versioning, SAS.

Azure Blob Storage is Microsoft's massively scalable object storage service for unstructured data — images, videos, log files, backups, and static website assets. It is the direct equivalent of AWS S3, storing objects (blobs) inside flat containers rather than a traditional hierarchical filesystem.

💡 Layman Explanation

Think of Blob Storage as an infinitely large warehouse of labelled boxes. Each box (blob) has a unique label (its path/name) inside a section (container), and you can retrieve any box instantly by its label — regardless of whether the warehouse holds 10 boxes or 10 billion.

12.1 What is Blob Storage?

📝 Theory & Key Points

- A **Storage Account** is the top-level namespace (globally unique name, e.g. stdevopslab001.blob.core.windows.net) that contains Blob, File, Queue, and Table services.
- A **Container** is a logical grouping of blobs, similar to an S3 bucket (but nested one level inside a Storage Account, unlike S3's flat global bucket namespace).
- Three blob types exist: **Block Blobs** (most common — text/binary files, uploaded in blocks, best for streaming/large files), **Append Blobs** (optimised for append-only operations like logging), and **Page Blobs** (random read/write, used as the backing store for VHD disks).
- Blob names can include forward slashes (e.g. images/2026/aug/photo.jpg) to simulate a folder hierarchy, though the underlying storage is a flat namespace unless Hierarchical Namespace (ADLS Gen2) is enabled.

AZURE TERM	AWS S3 EQUIVALENT
Storage Account	N/A (closest: an S3-compatible account boundary)
Container	Bucket
Blob	Object
Access Tier (Hot/Cool/Archive)	Storage Class (Standard/IA/Glacier)
Shared Access Signature (SAS)	Pre-signed URL
Lifecycle Management Policy	S3 Lifecycle Rule

12.2 Storage Classes (Access Tiers)

TIER	ACCESS PATTERN	COST PROFILE	MIN. STORAGE DURATION
Hot	Frequently accessed data	Highest storage cost, lowest access cost	None
Cool	Infrequently accessed, stored ≥30 days	Lower storage cost, higher access cost	30 days
Cold	Rarely accessed, stored ≥90 days	Lower than Cool, higher access cost than Cool	90 days
Archive	Rarely accessed, offline, stored ≥180 days	Lowest storage cost, rehydration required (hours) to read	180 days

12.3 Versioning

📝 Theory & Key Points

- When **Blob Versioning** is enabled on a Storage Account, every overwrite or delete automatically creates a new version rather than destroying data, letting you restore any previous version.
- Combine with **Soft Delete** (blob and container level) for a configurable retention window during which deleted blobs can be undeleted.
- **Point-in-Time Restore** can revert an entire container to an earlier state, useful for recovering from accidental bulk deletion or ransomware-style corruption (requires versioning + change feed + blob soft delete enabled).

12.4 Blob Container ACL and Permissions

📝 Theory & Key Points

- By default, all containers are **Private** — no anonymous access, every request must be authenticated (Storage Account key, Entra ID/RBAC, or a SAS token).
- Container-level public access can optionally be set to Blob (anonymous read of individual blobs if you know the exact URL) or Container (anonymous read + listing of all blobs) — generally discouraged for anything beyond public static website assets.
- **Shared Access Signatures (SAS)** grant time-limited, scope-limited access (e.g. read-only, expires in 1 hour) without sharing the account's master key — the standard way to give external users temporary access.
- **Azure RBAC** (e.g. Storage Blob Data Reader/Contributor roles) is the modern, identity-based way to grant access, preferred over shared account keys for auditability.

```bash
# Create a storage account and a container
az storage account create --resource-group rg-devops-lab --name stdevopslab001 \
  --sku Standard_LRS --kind StorageV2 --access-tier Hot

az storage container create --account-name stdevopslab001 --name uploads --auth-mode
login

# Upload a blob
az storage blob upload --account-name stdevopslab001 --container-name uploads \
  --name reports/aug2026.pdf --file ./aug2026.pdf --auth-mode login

# Enable versioning + soft delete on the storage account
az storage account blob-service-properties update --account-name stdevopslab001 \
  --enable-versioning true

az storage blob service-properties delete-policy update --account-name stdevopslab001
\
  --enable true --days-retained 14

# Generate a time-limited SAS URL (read-only, expires in 1 hour)
az storage blob generate-sas --account-name stdevopslab001 --container-name uploads \
  --name reports/aug2026.pdf --permissions r \
  --expiry $(date -u -d "+1 hour" '+%Y-%m-%dT%H:%MZ') --https-only -o tsv
```

🎯 Scenario-Based Interview Questions

Q: A partner company needs to download one specific invoice PDF from your private container for exactly the next 30 minutes, and must not be able to see or download anything else. How do you provide access? Generate a Shared Access Signature (SAS) scoped to that single blob, with the Read permission only, an expiry timestamp 30 minutes in the future, and ideally restricted to HTTPS and ideally the partner's known IP range for extra safety. Share only the resulting SAS URL — the storage account key itself is never exposed, the partner cannot list other blobs in the container (SAS is scoped to the one blob, not the container), and after 30 minutes the URL simply stops working with no manual cleanup required.

BLOBA

CHAPTER 13 · OBJECT STORAGE (BLOB)

Object Storage — Blob Advanced

Partitioning, AzCopy, replication (LRS/ZRS/GRS), lifecycle policies, Key Vault.

Beyond basic upload/download, production use of Blob Storage requires understanding how data is partitioned for performance, how to move data at scale, how replication protects against regional failure, how lifecycle policies automate cost optimisation, and how encryption and static website hosting round out the platform's capabilities.

13.1 Data Partitioning

📝 Theory & Key Points

- Azure Storage automatically load-balances traffic across partitions based on the **partition key**, which for Blob Storage is derived from the account name + container + blob name.
- Sequential naming patterns (e.g. timestamps as a prefix: 2026-08-16-log1.txt, 2026-08-16-log2.txt) can create a 'hot partition' bottleneck under very high request rates, since consecutive requests land on the same partition range.
- Best practice for extremely high-throughput workloads: add a random or hashed prefix (e.g. a hash of the blob name) at the start of the blob name to spread load across partitions — the same principle as S3 key-naming best practices.

13.2 Data Copying and Moving

BASH

```
# AzCopy — the high-performance tool for bulk copy/move (much faster than az storage
blob upload for large jobs)

# Copy an entire local folder to a container recursively
azcopy copy "./localfolder" "https://stdevopslab001.blob.core.windows.net/uploads?
<SAS>" --recursive

# Copy blobs between two storage accounts directly (server-side, no local download)
azcopy copy "https://srcaccount.blob.core.windows.net/data?<SAS>" \
  "https://destaccount.blob.core.windows.net/data?<SAS>" --recursive

# Sync a local folder with a container (mirrors changes, like rsync)
azcopy sync "./website" "https://stdevopslab001.blob.core.windows.net/\$web?<SAS>" --
delete-destination true
```

13.3 Cross-Region / Same-Region Replication

REDUNDANCY OPTION	COPIES	PROTECTS AGAINST	NOTES
LRS (Locally Redundant Storage)	3 copies within one datacenter	Hardware failure (disk/rack)	Cheapest, no protection against datacenter-level disaster
ZRS (Zone Redundant Storage)	3 copies across 3 Availability Zones in one region	Datacenter/zone-level outage	Data stays in-region
GRS (Geo Redundant Storage)	LRS in primary region + async-replicated LRS copy in a paired secondary region	Full regional disaster	Secondary copy not readable unless a failover is triggered
RA-GRS (Read-Access GRS)	Same as GRS	Full regional disaster + gives read access to secondary	Secondary region readable at all times via a `-secondary` endpoint
GZRS / RA-GZRS	ZRS in primary + geo-replicated copy in secondary	Zone failure AND regional disaster	Highest durability option

13.4 Lifecycle Management

📝 Theory & Key Points

- A **Lifecycle Management Policy** is a set of rules that automatically transitions blobs between access tiers or deletes them, based on age or last-modified/last-accessed date — fully automated cost optimisation.
- Common pattern: Hot for 30 days → auto-move to Cool → auto-move to Archive after 180 days → auto-delete after 7 years (for compliance retention, then cleanup).

JSON

```
{
  "rules": [
    {
      "name": "archive-old-logs",
      "enabled": true,
      "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["logs/"] },
        "actions": {
          "baseBlob": {
            "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
            "tierToArchive":{ "daysAfterModificationGreaterThan": 180 },
            "delete":        { "daysAfterModificationGreaterThan": 2555 }
          }
        }
      }
    }
  ]
}
```

13.5 Security & Encryption

📝 Theory & Key Points

- All data in Blob Storage is **encrypted at rest by default** using 256-bit AES (Storage Service Encryption) — this cannot be disabled.
- By default, Microsoft manages the encryption keys (**Microsoft-managed keys**). For stricter compliance, you can bring your own keys via **Customer-Managed Keys (CMK)** stored in Azure Key Vault, giving you rotation and revocation control.
- Data in transit is protected via HTTPS/TLS; the 'Secure transfer required' setting on a Storage Account rejects any unencrypted HTTP connection.

13.6 Azure Key Vault (KMS Equivalent)

📝 Theory & Key Points

- **Azure Key Vault** is the centralised secrets, keys, and certificates management service — the equivalent of AWS KMS + Secrets Manager combined.
- Stores three kinds of objects: **Secrets** (passwords, connection strings), **Keys** (cryptographic keys for encryption/signing, can be software or HSM-backed), and **Certificates** (TLS/SSL certs with auto-renewal).
- Applications retrieve secrets via a **Managed Identity** (no embedded credentials at all) rather than storing a Key Vault access key in code.

13.7 Static Website Hosting with Blob Storage

📝 Theory & Key Points

- Enabling the **Static Website** feature on a Storage Account creates a special `$web` container and exposes a public HTTPS endpoint capable of serving an index document and a custom 404 error page — ideal for hosting a React/Vue SPA or a simple static site cheaply.
- Typically paired with Azure CDN or Azure Front Door in front of it for a custom domain, caching, and global low-latency delivery.

BASH

```
# Enable static website hosting
az storage blob service-properties update --account-name stdevopslab001 \
  --static-website --index-document index.html --404-document 404.html

# Upload the site content to the special $web container
azcopy copy "./dist/*" "https://stdevopslab001.z13.web.core.windows.net/?<SAS>" --
recursive
```

13.8 Events Configuration on Blob Containers

📝 Theory & Key Points

- **Azure Event Grid** integrates natively with Blob Storage to publish events such as BlobCreated or BlobDeleted, letting you trigger serverless workflows (an Azure Function, Logic App, or webhook) the instant a file lands in a container.
- Common pattern: an uploaded image triggers an Event Grid event → an Azure Function subscribed to that event automatically generates a thumbnail and writes it back to a different container — a fully event-driven, serverless image-processing pipeline.

🎯 Scenario-Based Interview Questions

Q: Your company must retain financial transaction logs for 7 years for compliance, but only the last 30 days are ever actually queried, and cost is a major concern. Design a storage strategy. Store logs in a Blob container starting in the Hot tier for fast access during the first 30 days when they're actively queried. Configure a Lifecycle Management Policy to automatically transition blobs to Cool after 30 days (cheaper storage, occasional access) and to Archive after ~180 days (cheapest storage, hours-long rehydration if ever needed for an audit), then automatically delete at the 7-year (2555 day) mark once the compliance window closes. Enable GRS (or RA-GRS) redundancy for the regulatory requirement of surviving a regional disaster, and enable immutability policies (WORM — Write Once Read Many) if the compliance regime requires tamper-proof retention, preventing even an admin from deleting logs before the retention period expires.

BLOBX

CHAPTER 14 · OBJECT STORAGE (BLOB)

Object Storage — Cross-Account Blob Access

Cross-account and cross-subscription Blob access patterns.

Real-world architectures frequently need one team's application (in Subscription/Account A) to read or write data in another team's Storage Account (Subscription/Account B) — for example, a shared data lake consumed by multiple analytics teams. Azure provides several controlled mechanisms for this cross-account access.

14.1 Enabling Cross-Account Access for Blob

MECHANISM	HOW IT WORKS	BEST FOR
Shared Access Signature (SAS)	Time-boxed, permission-scoped token generated by the resource owner and handed to the consumer	One-off or external-partner access without granting a persistent identity role
Azure RBAC (Cross-Tenant/Subscription)	Assign a role (e.g. Storage Blob Data Reader) to a user, group, or Service Principal from another subscription/tenant on the target Storage Account/container	Ongoing, auditable access between teams within the same organisation
Storage Account Firewall + VNET Service Endpoint	Restrict the Storage Account to only accept traffic from specific VNETs/subnets or IP ranges, then grant RBAC to identities inside that network	Locking storage down to only trusted internal networks
Private Endpoint	Injects a private IP for the Storage Account directly inside a consumer's VNET, so traffic never traverses the public internet	Highest-security, compliance-sensitive cross-account access

BASH

```
# Grant a Service Principal from another subscription read access via RBAC
az role assignment create \
  --assignee <service-principal-object-id> \
  --role "Storage Blob Data Reader" \
  --scope /subscriptions/<target-sub-id>/resourceGroups/rg-
data/providers/Microsoft.Storage/storageAccounts/stsharedlake

# Restrict the storage account to only allow traffic from one VNET subnet
az storage account network-rule add --resource-group rg-data --account-name
stsharedlake \
  --vnet-name vnet-analytics --subnet snet-analytics

az storage account update --resource-group rg-data --account-name stsharedlake \
  --default-action Deny
```

🎯 Scenario-Based Interview Questions

Q: Team A owns a Storage Account with sensitive customer data. Team B (a different subscription, same company) needs read-only access for a nightly analytics job, and security requires that traffic never crosses the public internet. What do you configure? Set the Storage Account's network rules to Deny public access by default, then create a Private Endpoint for the Storage Account inside Team B's VNET — this gives the Storage Account a private IP address reachable only from within Team B's (and any peered) network, so traffic stays entirely on Microsoft's private backbone. Grant Team B's analytics job identity (ideally a Managed Identity, not a shared key) the 'Storage Blob Data Reader' RBAC role scoped only to the specific container it needs, satisfying least privilege alongside the network-level isolation.

ID

CHAPTER 15 · IDENTITY & ACCESS

Identity & Access — Microsoft Entra ID (Azure AD)

Microsoft Entra ID — MFA, RBAC, managed identities, auditing.

Microsoft Entra ID (formerly Azure Active Directory / Azure AD) is Azure's cloud-based identity and access management service — the system of record for every human user, group, and application (service principal) that can authenticate to Azure, and the engine behind Role-Based Access Control (RBAC).

💡 Layman Explanation

Entra ID is like the building's central security desk. Every person (user) and every robot courier (application/service principal) must badge in here first. The security desk doesn't just check identity — it also knows exactly which floors and rooms (Azure resources) each badge is allowed to enter, based on their assigned role.

15.1 Root Account vs. User

📝 Theory & Key Points

- Azure has no single 'root account' the way AWS does. Instead, the **Global Administrator** role in Entra ID has the broadest tenant-wide administrative rights, and the subscription's **Owner** role has the broadest rights over resources within that subscription.
- Best practice mirrors AWS: never use a Global Administrator or Owner account for daily work. Create individual named user accounts with least-privilege roles, and reserve break-glass admin accounts (with MFA and tightly monitored) for emergencies only.
- Every user, whether human or a guest from another organisation (B2B), authenticates against Entra ID before being evaluated against RBAC role assignments on any Azure resource.

15.2 Multi-Factor Authentication (MFA)

📝 Theory & Key Points

- MFA requires a second proof of identity beyond a password — typically the Microsoft Authenticator app, an SMS/phone call, or a FIDO2 security key.
- Enforced tenant-wide via **Conditional Access Policies** — e.g. 'require MFA for all users when signing in from outside the corporate network' or 'require MFA for all Global Administrators, always'.
- MFA is one of the single highest-impact security controls available — Microsoft's own data shows it blocks the overwhelming majority of account-compromise attacks, since a stolen password alone is no longer sufficient.

15.3 Password Policies

📝 Theory & Key Points

- Entra ID enforces baseline password complexity (length, character mix) and **Smart Lockout**, which locks an account after repeated failed attempts to blunt brute-force and password-spray attacks.
- **Self-Service Password Reset (SSPR)** lets users securely reset their own forgotten password via registered verification methods, reducing helpdesk load.
- Modern guidance (aligned with NIST) increasingly favours passwordless authentication (Windows Hello, FIDO2 keys, Authenticator app sign-in) over frequent forced password rotation, which research shows often leads to weaker, more predictable passwords.

15.4 Keys Configuration and Access Approaches

IDENTITY TYPE	USED BY	CREDENTIAL
User Account	A human logging into the Azure Portal/CLI	Password + MFA
Service Principal	An application or CI/CD pipeline needing to authenticate	Client ID + Client Secret, or Certificate
Managed Identity (System-Assigned)	A single Azure resource (e.g. one VM) needing to call other Azure services	No credential at all — Azure manages it automatically, tied to that resource's lifecycle
Managed Identity (User-Assigned)	Shared across multiple resources needing the same identity	No credential — created as a standalone Azure resource, assigned to many VMs/Functions

⚠️ Important Warning

- Managed Identities are strongly preferred over Service Principal client secrets wherever possible — there is no secret to leak, rotate, or expire.
- Never hardcode a Service Principal's client secret in source code or a pipeline YAML file in plaintext — store it in Azure Key Vault and reference it, or use OIDC federated credentials (no secret at all) for CI/CD like GitHub Actions/Azure DevOps.

15.5 Users, Groups, and Role Concepts (RBAC)

📝 Theory & Key Points

- **Azure RBAC** grants access by assigning a **Role** (a collection of permissions) to a **Security Principal** (user, group, service principal, or managed identity) at a specific **Scope** (Management Group → Subscription → Resource Group → Resource).
- Permissions are additive — a user's effective access is the union of every role assignment at every scope above and at the resource itself. There is no explicit 'Deny' in classic Azure RBAC (Azure does support explicit Deny assignments for advanced scenarios, but Allow-only is the default mental model).
- Always assign roles to **Groups**, not individual users — when someone joins or leaves a team, you add/remove their group membership once instead of hunting down dozens of individual resource-level assignments.
- Built-in roles follow least-privilege granularity — e.g. **Reader** (view only), **Contributor** (manage resources but not grant access to others), **Owner** (full control including granting access), plus hundreds of service-specific roles like **Storage Blob Data Contributor**.

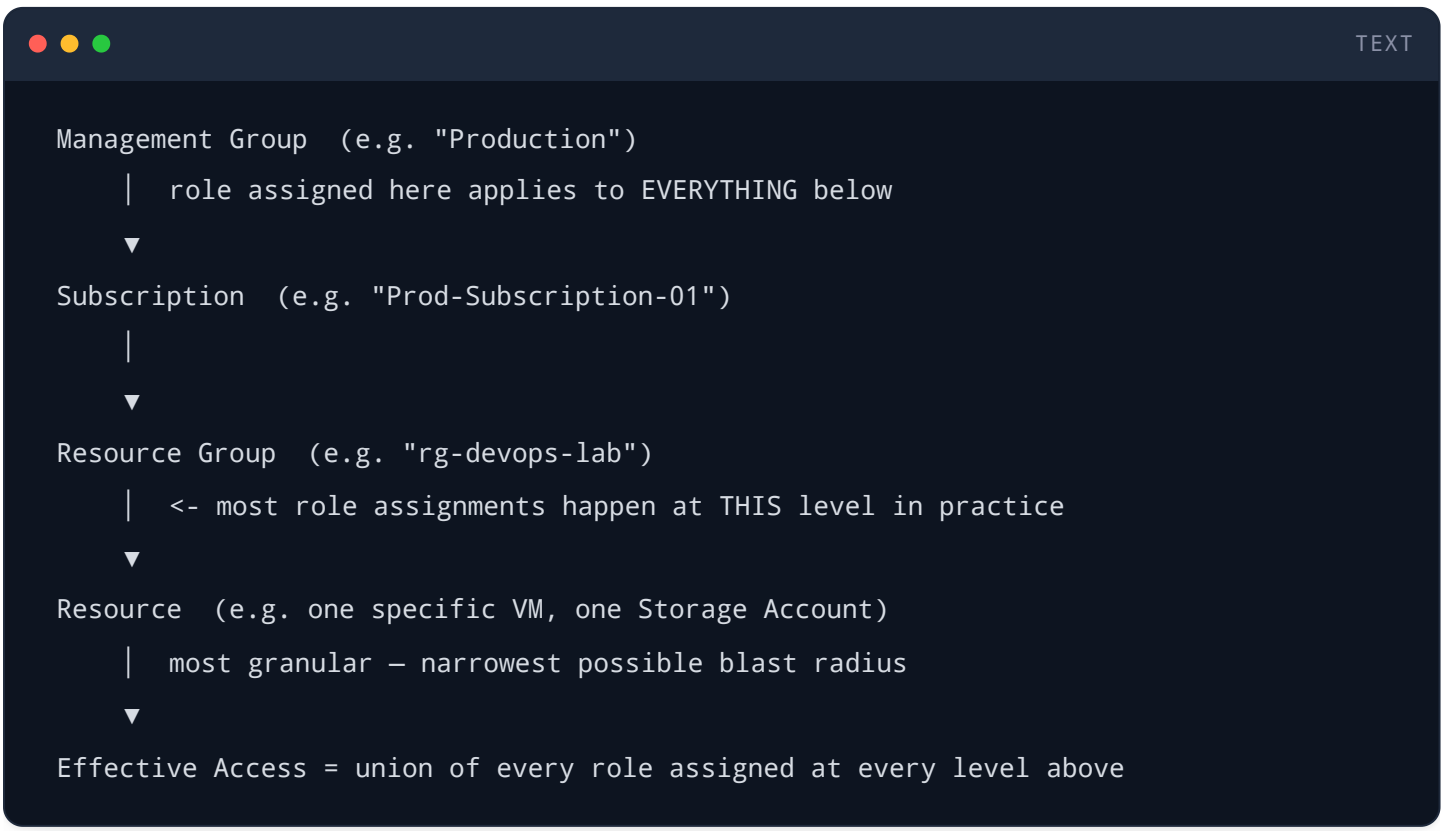


Fig 7: Azure RBAC Scope Hierarchy — Permissions Inherit Downward

15.6 Creating Managed (Custom) Policies

BASH

```
# Create a Microsoft Entra security group
az ad group create --display-name "DevOps-Engineers" --mail-nickname "devops-
engineers"

# Assign the built-in 'Contributor' role to that group, scoped to one resource group
az role assignment create \
  --assignee-object-id <group-object-id> --assignee-principal-type Group \
  --role "Contributor" \
  --scope /subscriptions/<sub-id>/resourceGroups/rg-devops-lab

# Create a custom role: allow starting/restarting VMs but NOT deleting them
cat > custom-role.json <<'JSONEOF'
{
  "Name": "VM Operator - No Delete",
  "IsCustom": true,
  "Actions": [
    "Microsoft.Compute/virtualMachines/start/action",
    "Microsoft.Compute/virtualMachines/restart/action",
    "Microsoft.Compute/virtualMachines/read"
  ],
  "NotActions": [],
  "AssignableScopes": ["/subscriptions/<sub-id>"]
}
JSONEOF
az role definition create --role-definition custom-role.json

# Enable a system-assigned managed identity on a VM, no credentials needed
az vm identity assign --resource-group rg-devops-lab --name vm-app01
```

15.7 Auditing User Activity

📝 Theory & Key Points

- The **Azure Activity Log** is an automatic, tenant-wide audit trail of every control-plane operation (who created, modified, or deleted which resource, when, and from which IP) — retained 90 days by default, exportable to a Log Analytics Workspace for longer retention.
- The **Entra ID Sign-in Logs** and **Audit Logs** separately track authentication events (successful/failed logins, MFA challenges) and identity-management operations (role assignments, group membership changes).
- **Microsoft Defender for Cloud** and **Microsoft Sentinel** (Azure's SIEM) can correlate these logs to detect anomalous activity, such as impossible-travel logins or privilege escalation attempts.

🎯 Scenario-Based Interview Questions

Q: A contractor needs to deploy and manage VMs in exactly one resource group for a 3-month engagement, but must never be able to view or touch the billing/subscription-level settings, and must never be able to grant access to anyone else. What role and scope do you assign? Assign the built-in **Contributor** role — not Owner — scoped narrowly to just that one Resource Group, not the Subscription. Contributor allows full management of resources within scope (create/modify/delete VMs, disks, networking) but explicitly cannot grant or modify role assignments (that requires Owner or User Access Administrator) and has no visibility into subscription billing. Set an expiration on the role assignment via Privileged Identity Management (PIM) if available, so access automatically lapses at the end of the 3-month engagement without requiring anyone to remember to revoke it manually.

Q: You're auditing an incident where a production VM was deleted without authorisation. How do you trace what happened using Azure's native tools? Query the Azure Activity Log (via Azure Portal → Monitor → Activity Log, or KQL against the Log Analytics Workspace it's exported to) filtered by the resource ID of the deleted VM and the 'Delete Virtual Machine' operation — this reveals exactly which identity (user or service principal) performed the delete, the timestamp, and the source IP. Cross-reference that identity against the Entra ID Sign-in Logs around that timestamp to confirm it was a legitimate authenticated session (and check for MFA) rather than a compromised credential. If access should never have been possible, review the RBAC role assignments at that scope to find and correct the over-permissioning that allowed it.

SQL

CHAPTER 16 · DATABASES

Databases — Azure SQL Database

Azure SQL Database — PaaS vs IaaS, connection proxy modes, Redis cache.

Azure SQL Database is a fully managed Platform-as-a-Service (PaaS) relational database, built on the Microsoft SQL Server engine, that removes the operational burden of patching, backups, and high-availability configuration — you focus purely on schema design and queries.

16.1 Launching an Azure SQL Instance

BASH

```
# Create a logical SQL Server (a management/auth boundary, not a VM)
az sql server create --resource-group rg-devops-lab --name sqlsrv-devopslab \
  --admin-user sqladmin --admin-password "P@ssw0rd12345!"

# Allow your current IP through the server-level firewall
az sql server firewall-rule create --resource-group rg-devops-lab \
  --server sqlsrv-devopslab --name AllowMyIP \
  --start-ip-address 203.0.113.10 --end-ip-address 203.0.113.10

# Create the actual database on that logical server, General Purpose tier
az sql db create --resource-group rg-devops-lab --server sqlsrv-devopslab \
  --name appdb --service-objective GP_Gen5_2 --backup-storage-redundancy Zone
```

16.2 Difference Between DB on VM vs Managed Azure SQL

ASPECT	SQL SERVER ON AZURE VM (IAAS)	AZURE SQL DATABASE (PAAS)
OS/Patching	You manage Windows/Linux + SQL Server patching	Fully managed by Microsoft
Backups	You configure and manage backup jobs	Automatic, continuous, point-in-time restore built in
High Availability	You build it (Always On Availability Groups, clustering)	Built-in, transparent (zone-redundant options available)
Control	Full OS/instance-level access, custom SQL Server features, agent jobs	No OS access; some legacy SQL Server features unavailable
Best for	Lift-and-shift of legacy apps needing full SQL Server surface area/agent jobs	New/cloud-native apps wanting minimal operational overhead
Scaling	Manual — resize the VM, reconfigure storage	Change service tier/vCores via a single command, often online

16.3 Proxy Configuration — Connection Process

📝 Theory & Key Points

- Azure SQL Database connections are always routed through a **Gateway** — by default, using **Proxy mode**, all traffic (both the initial login and subsequent data traffic) flows through the Azure SQL Gateway on port 1433.
- **Redirect mode** (recommended for apps inside Azure, e.g. an App Service or VM in the same region) has the client establish the login via the gateway but then redirect to connect directly to the database node, reducing latency — requires additional outbound ports (11000-11999) to be open.
- For apps outside Azure or behind restrictive corporate firewalls that can't open the redirect port range, Proxy mode (port 1433 only) is used as the compatible fallback.

16.4 Azure Cache for Redis (ElastiCache Equivalent)

📝 Theory & Key Points

- **Azure Cache for Redis** is a fully managed, in-memory data store used to reduce database load and slash read latency from milliseconds to microseconds — the direct equivalent of AWS ElastiCache for Redis.
- Common patterns: caching frequent, expensive SQL query results; storing user session state for a horizontally-scaled web tier; implementing a distributed lock or rate limiter; acting as a pub/sub message bus for lightweight real-time features.
- Tiers range from Basic (single node, dev/test, no SLA) to Standard (replicated pair with automatic failover) to Premium (clustering, persistence, VNET injection, higher throughput) to Enterprise (Redis Enterprise features like modules and active geo-replication).

BASH

```
# Create a Standard-tier Redis cache
az redis create --resource-group rg-devops-lab --name redis-devopslab \
  --sku Standard --vm-size C1 --location centralindia

# Get the access key needed by the application to connect
az redis list-keys --resource-group rg-devops-lab --name redis-devopslab
```

🎯 Scenario-Based Interview Questions

Q: A read-heavy product catalog page hits the SQL database on every page load, and the DB is now the bottleneck at peak traffic. The catalog data changes only a few times a day. What's the fix? Introduce Azure Cache for Redis as a read-through cache in front of the database: on a page load, the application first checks Redis for the catalog data; on a cache miss it queries Azure SQL Database once, then writes the result into Redis with a TTL (e.g. 15 minutes, or invalidated explicitly whenever the catalog is updated). Since the catalog changes only a few times daily, the cache hit rate will be extremely high, cutting the vast majority of read traffic away from the database entirely and moving it to Redis's sub-millisecond in-memory lookups — the database is then free to handle writes and less-cacheable queries.

SQLOP

CHAPTER 17 · DATABASES

Databases — Azure SQL Operations

Connecting to Azure SQL, running queries, and read replicas.

Day-to-day work with Azure SQL Database involves connecting from application servers and local developer machines, running everyday DDL/DML, and configuring read replicas for reporting workloads or read-scale needs — all without ever touching the underlying OS.

17.1 Connecting from an Azure VM

BASH

```
# From inside an Azure VM in the same region — first ensure the VM's outbound IP
# (or VNET, via a Private Endpoint) is allowed through the SQL Server firewall
az sql server firewall-rule create --resource-group rg-devops-lab \
  --server sqlsrv-devopslab --name AllowAppVM \
  --start-ip-address <vm-public-ip> --end-ip-address <vm-public-ip>

# Connect using sqlcmd from the VM
sqlcmd -S sqlsrv-devopslab.database.windows.net -d appdb -U sqladmin -P
"P@sswOrd12345!" -N -C
```

17.2 Connecting from a Local Machine via Workbench/SSMS

📝 Theory & Key Points

- From a laptop, first add the laptop's public IP to the SQL Server firewall (via Portal or CLI) — Azure SQL denies all connections by default until an explicit rule is added.
- Connect using SQL Server Management Studio (SSMS) on Windows, Azure Data Studio (cross-platform), or MySQL Workbench-style GUI tools using the server name `<server>.database.windows.net`, port 1433, with SQL or Entra ID authentication.
- For production, prefer Entra ID authentication over SQL authentication (username/password) — it enables MFA and centralised access revocation through the same identity system as everything else in Azure.

17.3 Creating DBs, Tables, and Running Queries

```sql
-- Create a table
CREATE TABLE Orders (
    OrderId      INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId   INT NOT NULL,
    OrderDate    DATETIME2 DEFAULT SYSUTCDATETIME(),
    Status       VARCHAR(20) DEFAULT 'PENDING',
    TotalAmount  DECIMAL(10,2) NOT NULL
);

-- Insert a row
INSERT INTO Orders (CustomerId, TotalAmount) VALUES (1042, 249.99);

-- Query with an index-friendly filter
SELECT OrderId, TotalAmount, Status
FROM Orders
WHERE CustomerId = 1042 AND OrderDate > DATEADD(day, -30, SYSUTCDATETIME());

-- Create a non-clustered index to speed up the above query
CREATE NONCLUSTERED INDEX IX_Orders_CustomerId_OrderDate
ON Orders (CustomerId, OrderDate);
```

17.4 Configuring Read Replicas (Read Scale-Out)

📝 Theory & Key Points

- **Read Scale-Out** provisions up to 4 free, geo-replicated (or zone-replicated) read-only replicas of a Premium/Business Critical tier database, automatically kept in sync — reporting/analytics queries can be routed to a replica so they never compete with production write traffic.
- Applications opt in by adding `ApplicationIntent=ReadOnly` to their connection string; Azure SQL automatically routes that connection to a replica.
- For cross-region disaster recovery (not just read scaling), **Active Geo-Replication** or **Auto-Failover Groups** create a readable secondary database in a different Azure region, which can be manually or automatically promoted to primary during a regional outage.

BASH

```
# Enable an Auto-Failover Group with a secondary server in another region
az sql server create --resource-group rg-devops-lab --name sqlsrv-devopslab-dr \
  --admin-user sqladmin --admin-password "P@ssw0rd12345!" --location eastus

az sql failover-group create --name fg-appdb --resource-group rg-devops-lab \
  --server sqlsrv-devopslab --partner-server sqlsrv-devopslab-dr \
  --add-db appdb --failover-policy Automatic --grace-period 1
```

🎯 Scenario-Based Interview Questions

Q: The reporting team's nightly dashboard queries are so heavy they're causing timeouts for real production checkout transactions on the same database. Your fix must not require standing up a separate database or ETL pipeline. Enable Read Scale-Out on the existing Azure SQL Database (requires Premium or Business Critical tier) and point the reporting team's connection string at it with `ApplicationIntent=ReadOnly`. Their heavy analytical queries are then automatically routed to a synchronised read-only replica, isolating that load away from the primary replica that handles checkout writes — all without provisioning any new database, running an ETL job, or changing the reporting queries themselves, since the replica is schema-identical and near-real-time.

APP

CHAPTER 18 · SERVERLESS

Serverless — Azure App Service & Functions

Azure App Service and Functions — PaaS web apps and FaaS.

Serverless computing lets you run code without provisioning or managing any servers — the cloud provider handles scaling, patching, and infrastructure entirely, and (for Functions) you often pay only for actual execution time. Azure offers two flagship serverless-adjacent compute services: App Service (for full web apps/APIs) and Azure Functions (for event-driven, granular code).

💡 Layman Explanation

Running your own VM is like owning a restaurant — you handle the building, the ovens, the electricity, staffing shifts, even when no customers show up. Serverless is like a cloud kitchen: you just supply the recipe (your code); the kitchen appears the instant an order (a request/event) comes in, cooks it, and disappears — you're billed per dish made, not per hour the kitchen sits idle.

18.1 Overview of Azure App Services

📝 Theory & Key Points

- **Azure App Service** is a fully managed PaaS for hosting web apps, REST APIs, and mobile backends in numerous languages (.NET, Java, Node.js, Python, PHP, Ruby) or as a custom Docker container, without managing the underlying VM/OS.
- Runs on an **App Service Plan** — the underlying compute tier (a set of VMs) that one or more App Service apps share; scaling the plan up/out scales every app on it.
- Built-in features: deployment slots (blue-green deployments), auto-scaling, custom domains + free managed TLS certificates, and continuous deployment integration straight from GitHub/Azure DevOps.

18.2 App Service Plan Tiers

TIER	CAPABILITIES	USE CASE
Free / Shared	Shared compute, no custom domain/SSL, limited quota	Learning, prototypes
Basic	Dedicated VM, custom domain + SSL, manual scale	Small production apps
Standard	Adds autoscale, staging slots, daily backups	Standard production workloads
Premium	Higher performance, more slots, VNET integration	High-traffic production apps
Isolated	Dedicated App Service Environment (ASE) inside your own VNET	Highly regulated/high-security workloads

18.3 Creation and Automation Using Azure Functions

📝 Theory & Key Points

- **Azure Functions** is Azure's Function-as-a-Service (FaaS) offering — the equivalent of AWS Lambda — running small, single-purpose pieces of code (a 'function') in response to a **trigger** (HTTP request, timer, queue message, blob upload, database change).
- On the **Consumption plan**, you pay only per execution (per-invocation + execution time in GB-seconds) and Azure scales instance count from zero to thousands automatically — the purest serverless economics.
- The **Premium plan** adds pre-warmed instances (eliminating 'cold start' latency) and VNET connectivity, while the **App Service Plan** option runs Functions on dedicated, always-on compute for predictable, latency-sensitive workloads.

```bash
# Create a Function App on the Consumption (serverless, pay-per-execution) plan
az functionapp create --resource-group rg-devops-lab --name func-devopslab \
  --consumption-plan-location centralindia --runtime python \
  --runtime-version 3.11 --functions-version 4 \
  --storage-account stdevopslab001

# Deploy local function code as a zip package
func azure functionapp publish func-devopslab

# Create an App Service Plan + Web App for a full REST API
az appservice plan create --resource-group rg-devops-lab --name plan-api \
  --sku S1 --is-linux

az webapp create --resource-group rg-devops-lab --name webapp-orders-api \
  --plan plan-api --runtime "PYTHON:3.11"
```

18.4 General Configuration and Environment Setup

📝 Theory & Key Points

- **Application Settings** (environment variables) and **Connection Strings** are configured outside the code, injected at runtime — never hardcode secrets in the codebase.
- For anything sensitive, reference an Azure Key Vault secret directly from an App Setting using the syntax `@Microsoft.KeyVault(SecretUri=...)`, combined with the App's Managed Identity for credential-free retrieval.
- **Deployment Slots** (e.g. 'staging') let you deploy and fully test a new version on a separate URL, then perform a **swap** — an instant, zero-downtime cutover to production that can also be instantly rolled back by swapping again.

18.5 Layers Equivalent and Importance

📝 Theory & Key Points

- AWS Lambda's **Layers** concept (shared libraries/dependencies reused across multiple functions without bundling them into every deployment package) maps in Azure Functions to shared code via a referenced NuGet/npm/pip package, a custom Docker base image, or by placing common code in a linked **Azure Functions Extension Bundle** / shared library project referenced by multiple Function Apps.
- For containerised Functions, the equivalent pattern is a shared base Docker image containing common dependencies, which individual function images then extend — keeping each function's deployable artifact small and consistent.
- The core benefit is the same as Lambda Layers: avoid duplicating large dependencies across every function, speed up deployment, and centralise version updates to shared code.

🎯 Scenario-Based Interview Questions

Q: You're deploying a new version of a critical API and management is nervous about downtime or a bad release. What Azure App Service feature directly addresses this, and how does it work? Use Deployment Slots. Deploy the new version to a 'staging' slot (which has its own distinct URL, fully isolated from production), run smoke tests and validation against it, then perform a slot swap — Azure re-routes traffic so 'staging' becomes 'production' and vice versa, essentially instantaneously and with warm-up already completed, avoiding a cold-start penalty on the swap. If anything goes wrong post-swap, you swap back immediately for an equally fast rollback, all without ever taking the app offline or requiring a new deployment to revert.

APPI

CHAPTER 19 · SERVERLESS

Serverless — App Service Integrations & Limitations

Function limitations and integrations — SQL, Cosmos DB, API Management.

Serverless compute rarely stands alone — Azure Functions is most powerful when wired directly into other managed services, letting you build event-driven pipelines with almost no custom plumbing code. This section covers connecting Functions to Azure SQL, Cosmos DB, and API Management, plus the practical limitations to plan around.

19.1 Limitations of Azure Functions

📝 Theory & Key Points

- **Cold starts:** on the Consumption plan, a function that hasn't run recently must 'spin up' a new instance on the next request, adding latency (typically hundreds of ms to a few seconds) — mitigated by the Premium plan's pre-warmed instances.
- **Execution time limits:** Consumption plan functions default to a 5-minute timeout (max 10 minutes); long-running workflows need **Durable Functions** (an orchestration extension) or should move to the Premium/Dedicated plan.
- **Stateless by design:** a function instance can be recycled at any time, so it must not rely on in-memory state persisting between invocations — any state needed must live in an external store (Azure SQL, Cosmos DB, Redis, Storage).
- **Local disk is ephemeral:** the temporary filesystem given to a function instance is not guaranteed to persist or be shared across instances.

19.2 Azure Functions to Azure SQL Connection

```bash
# App Setting holding the connection string (ideally referencing Key Vault, not plaintext)
az functionapp config appsettings set --resource-group rg-devops-lab --name func-devopslab \
  --settings "SqlConnectionString=@Microsoft.KeyVault(SecretUri=https://kv-devopslab.vault.azure.net/secrets/sql-conn-string/)"
```

```python
# Python function using the injected connection string (pseudocode)
import pyodbc, os
conn = pyodbc.connect(os.environ["SqlConnectionString"])
cursor = conn.cursor()
cursor.execute("INSERT INTO Orders (CustomerId, TotalAmount) VALUES (?, ?)", 1042, 249.99)
conn.commit()
```

19.3 Azure Functions to Cosmos DB and API Management

📝 Theory & Key Points

- **Cosmos DB Trigger:** a function can be triggered automatically whenever documents are inserted or updated in a Cosmos DB container's Change Feed — ideal for real-time reactive pipelines (e.g. re-index a search service the instant a product document changes), with no polling required.
- **Cosmos DB Input/Output Bindings:** declaratively read from or write to Cosmos DB without hand-writing SDK boilerplate — you just declare the binding in the function configuration.
- **API Management (APIM)** sits in front of Functions (and any backend API) as a managed API gateway, providing a single, versioned, documented, and secured entry point — handling API keys/OAuth validation, rate limiting/throttling, request/response transformation, and analytics, while the Function stays focused purely on business logic.

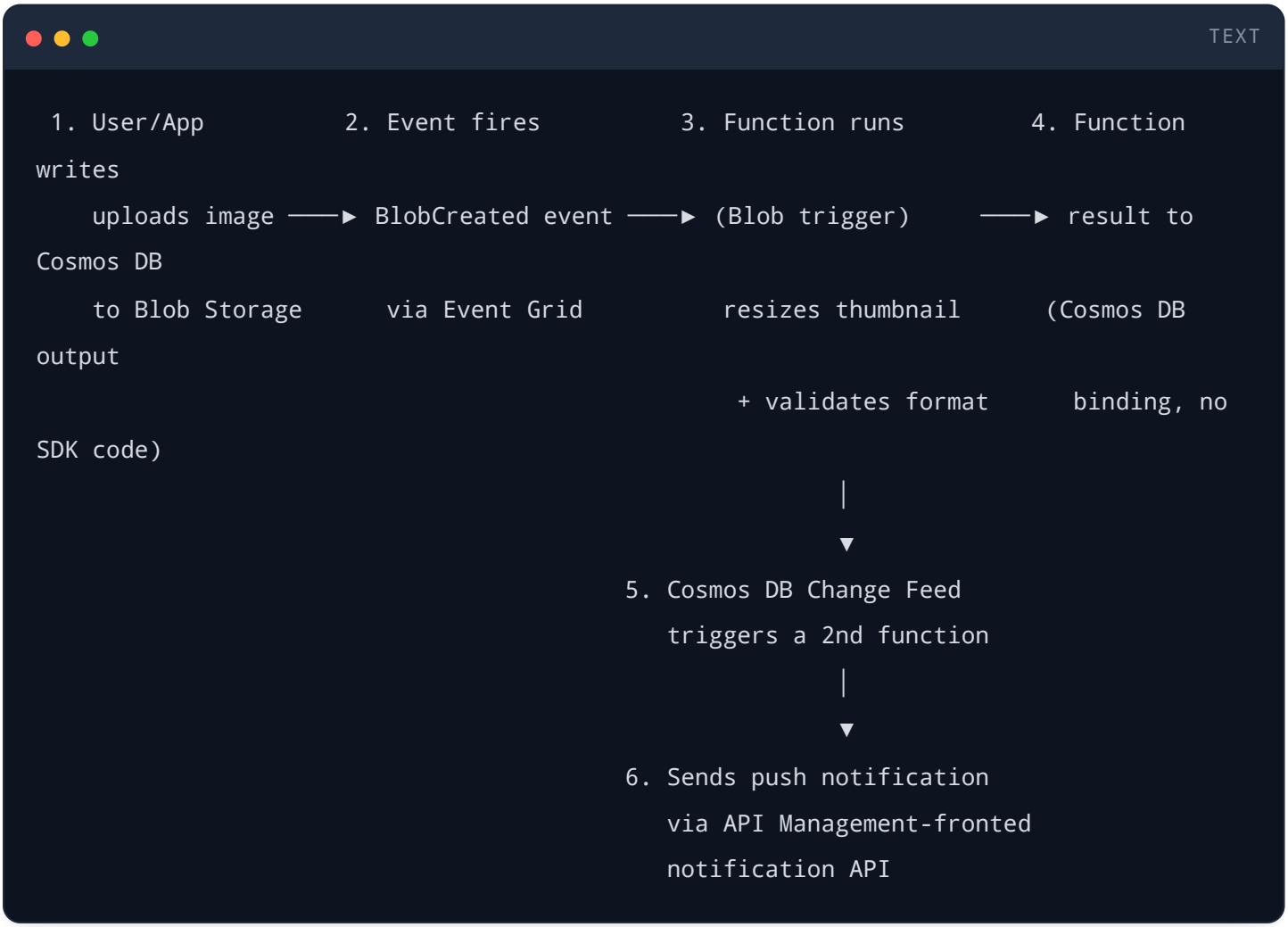


Fig 8: Event-Driven Serverless Pipeline — Blob Upload to Notification

🎯 Scenario-Based Interview Questions

Q: You want external partners to call your Functions-based pricing API, but need to enforce per-partner rate limits, require API keys, and hide the underlying Function App URL entirely. What do you add in front of the Functions? Place Azure API Management (APIM) in front of the Function App as the sole public entry point. Configure APIM policies for rate limiting per subscription key (per-partner quotas), require an API key or OAuth token on every incoming request, and set up URL rewriting so callers only ever see the APIM gateway's domain — the actual Function App's default hostname is never exposed and can even be locked down (via VNET integration or IP restrictions) to only accept traffic originating from APIM, giving both access control and a clean abstraction layer.

CICD

CHAPTER 20 · CI/CD PIPELINES

CI/CD Pipelines — Azure DevOps

Azure DevOps Pipelines — stages, YAML, deployment strategies.

Azure DevOps is Microsoft's suite of development collaboration tools — Boards (work tracking), Repos (Git hosting), Pipelines (CI/CD), Test Plans, and Artifacts (package feeds). For DevOps engineering, Azure Pipelines is the centerpiece: it automates building, testing, and deploying code every time a change is pushed.

💡 Layman Explanation

A CI/CD pipeline is an assembly line for software. Continuous Integration (CI) is the quality-control station: every time someone adds a part (commits code), it's automatically tested and inspected before it's allowed onto the line. Continuous Delivery/Deployment (CD) is the shipping station: once a part passes inspection, it's automatically boxed and shipped (deployed) to the customer (production) — no manual, error-prone hand-carrying required.

20.1 Core Azure Pipelines Concepts

📝 Theory & Key Points

- A **Pipeline** is defined as code in a YAML file (azure-pipelines.yml) checked into the repository — versioned and reviewable exactly like application code.
- A pipeline is made of **Stages** (e.g. Build, Test, Deploy-Dev, Deploy-Prod) → each Stage has **Jobs** (units of work that can run in parallel or on different agents) → each Job has sequential **Steps** (individual tasks/scripts).
- An **Agent** is the compute (VM or container) that actually executes the pipeline's steps — Microsoft-hosted agents are ephemeral and maintained by Azure; self-hosted agents run on your own VM/on-prem machine for custom tooling or network access needs.
- **Approvals and Gates** let you require a human sign-off (e.g. a release manager approving a production deployment) or an automated check (e.g. no active Sev-1 incidents) before a stage proceeds.
- **Variable Groups** and integration with Azure Key Vault let pipelines securely consume secrets without ever printing them in logs.

20.2 Example CI/CD Pipeline (YAML)

YAML

```yaml
trigger:
  branches:
    include: [ main ]

pool:
  vmImage: 'ubuntu-latest'

stages:
- stage: Build
  jobs:
  - job: BuildAndTest
    steps:
    - script: pip install -r requirements.txt
      displayName: 'Install dependencies'
    - script: pytest --junitxml=results.xml
      displayName: 'Run unit tests'
    - task: PublishTestResults@2
      inputs:
        testResultsFiles: 'results.xml'
    - task: Docker@2
      inputs:
        command: buildAndPush
        repository: myregistry.azurecr.io/orders-api
        tags: $(Build.BuildId)

- stage: DeployDev
  dependsOn: Build
  jobs:
  - deployment: DeployToDev
    environment: 'dev'
    strategy:
      runOnce:
        deploy:
          steps:
          - script: az webapp config container set --name webapp-orders-api-dev \
```

```
                --resource-group rg-devops-lab \
                --container-image-name myregistry.azurecr.io/orders-
api:$(Build.BuildId)

- stage: DeployProd
  dependsOn: DeployDev
  jobs:
  - deployment: DeployToProd
    environment: 'production'   # environment has an approval gate configured
    strategy:
      runOnce:
        deploy:
          steps:
          - script: az webapp config container set --name webapp-orders-api \
                --resource-group rg-devops-lab \
                --container-image-name myregistry.azurecr.io/orders-
api:$(Build.BuildId)
```

20.3 CI/CD Strategy Comparison

STRATEGY	HOW IT WORKS	TRADE-OFF
Blue-Green Deployment	Two identical environments; traffic switches entirely from old (blue) to new (green) once validated	Instant rollback (switch back), but requires 2x infrastructure during cutover
Canary Deployment	New version rolled out to a small % of traffic first, gradually increased if healthy	Lower blast-radius risk, but more complex traffic-splitting setup
Rolling Deployment	Instances updated in small batches, old version drained as new comes up	No extra infrastructure needed, but rollback is slower (must roll back the same way)

🎯 Scenario-Based Interview Questions

Q: Your pipeline currently deploys straight to production on every merge to main, and last week a bad deploy caused a 20-minute outage before anyone caught it. What pipeline changes reduce this risk without slowing the team down too much? Insert an automated Dev/Staging deployment stage with smoke tests and health checks before Prod, so bad code is caught before it ever reaches customers, plus a manual Approval Gate on the Production stage so a second person consciously signs off on the specific build being released (this doesn't block frequent deploys, it just adds one deliberate checkpoint). Combine this with adopting a Blue-Green or Canary strategy for the production stage itself, so even an approved-but-flawed release only affects a slice of traffic (canary) or can be instantly reverted with a single traffic-switch (blue-green) — turning a 20-minute outage into a near-zero-impact rollback.

AKS

CHAPTER 21 · ORCHESTRATION (AKS)

Orchestration — Azure Kubernetes Service (AKS)

Azure Kubernetes Service — control plane, node pools, autoscaling.

Azure Kubernetes Service (AKS) is Azure's fully managed Kubernetes offering — Microsoft operates and patches the Kubernetes control plane (API server, etcd, scheduler) for free, while you manage and pay only for the worker node VMs that run your containers. It is the standard way to run containerised applications at scale on Azure.

💡 Layman Explanation

If a single container is one shipping container, Kubernetes is the entire port authority — deciding which ship (node) each container goes on, rerouting containers if a ship sinks (a node fails), and automatically calling in more ships when the port gets busy (autoscaling). AKS means Microsoft runs the port authority's control tower for you; you just bring the containers.

21.1 Core AKS Architecture

📝 Theory & Key Points

- The **Control Plane** (API server, scheduler, etcd, controller manager) is fully managed by Azure at no direct cost on most tiers — you never see or patch these VMs.
- **Node Pools** are groups of VMs (using Azure VM Scale Sets under the hood) that actually run your Pods — you can have multiple node pools with different VM sizes (e.g. a cheap B-series pool for general workloads, a GPU N-series pool for ML jobs).
- A **Pod** is the smallest deployable unit — one or more tightly-coupled containers sharing network/storage. A **Deployment** manages a desired number of replica Pods and handles rolling updates. A **Service** provides a stable network endpoint (ClusterIP, NodePort, or LoadBalancer type) to reach a set of Pods even as individual Pod IPs change.
- The **Cluster Autoscaler** automatically adds/removes nodes based on unschedulable Pods, while the **Horizontal Pod Autoscaler (HPA)** scales the number of Pod replicas based on CPU/memory/custom metrics — the two work together for full elastic scaling.

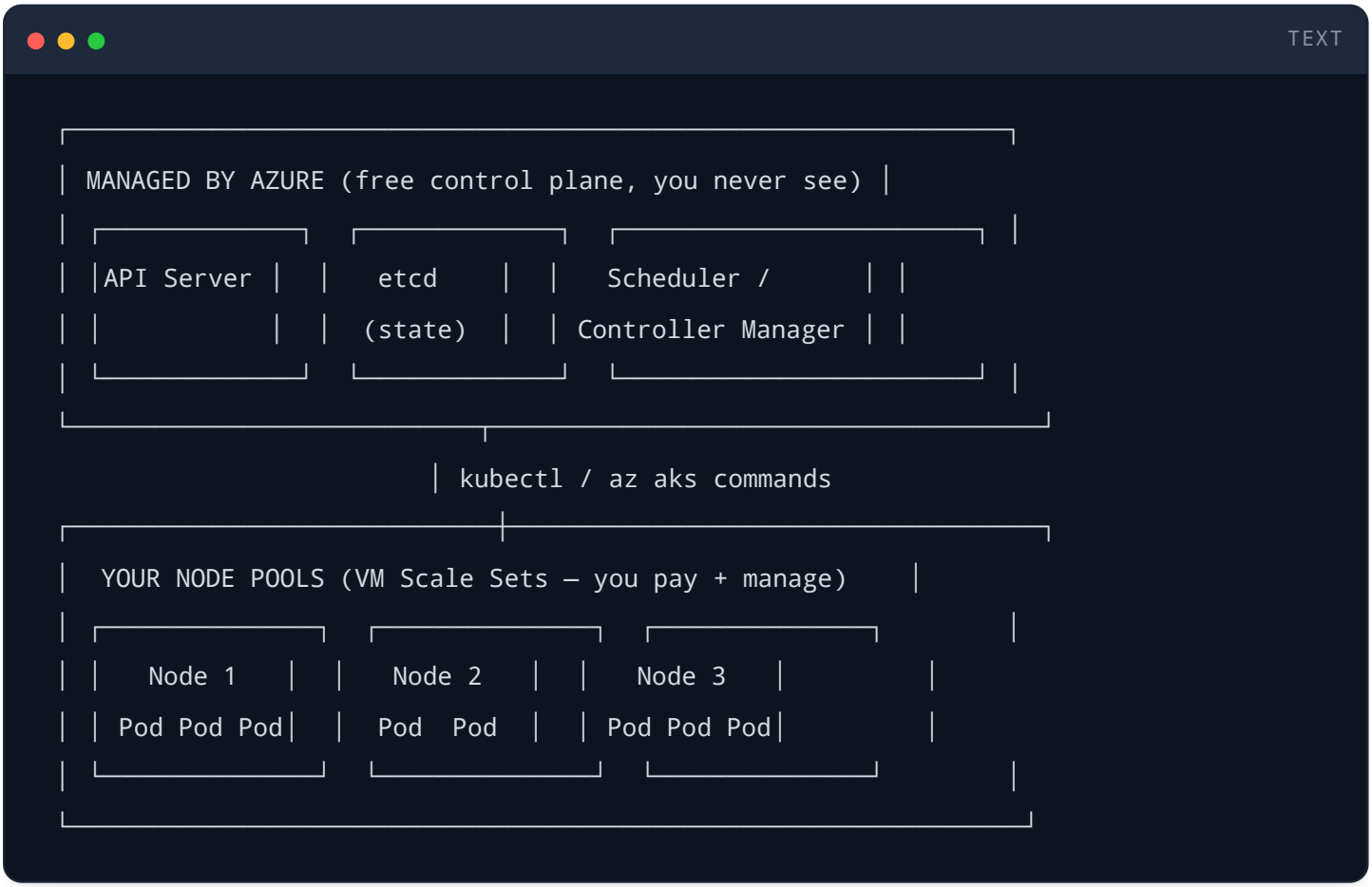


Fig 9: AKS Architecture — Managed Control Plane + Your Node Pools

21.2 AKS Overview and Deployment

BASH

```
# Create an AKS cluster with autoscaling enabled, 2-6 nodes
az aks create --resource-group rg-devops-lab --name aks-devopslab \
  --node-count 2 --min-count 2 --max-count 6 --enable-cluster-autoscaler \
  --node-vm-size Standard_D2s_v5 --generate-ssh-keys \
  --attach-acr myregistry

# Get cluster credentials so kubectl can talk to it
az aks get-credentials --resource-group rg-devops-lab --name aks-devopslab

# Deploy an application
kubectl create deployment orders-api --image=myregistry.azurecr.io/orders-api:latest --replicas=3

# Expose it via a LoadBalancer Service (provisions an Azure Load Balancer automatically)
kubectl expose deployment orders-api --type=LoadBalancer --port=80 --target-port=8080

# Configure Horizontal Pod Autoscaling — 3 to 10 pods, target 70% CPU
kubectl autoscale deployment orders-api --cpu-percent=70 --min=3 --max=10

# Watch pods scale in real time
kubectl get hpa -w
```

⚠️ Important Warning

- Always set resource requests/limits on every Pod — without them, the scheduler can't make good bin-packing decisions and one runaway container can starve every other Pod on the same node.
- Never run production workloads on the default node pool alone without a defined upgrade/patching strategy — plan node image upgrades (`az aks nodepool upgrade`) as a routine, tested operation.

🎯 Scenario-Based Interview Questions

Q: Your AKS cluster's Horizontal Pod Autoscaler wants to add more replicas to handle a traffic spike, but the pods stay stuck in 'Pending' state. What's likely happening and how do you fix it? 'Pending' Pods that HPA created but the scheduler can't place usually means there's no node in the cluster with enough free CPU/memory to satisfy the Pod's resource requests. Check with `kubectl describe pod <pod-name>` for a 'FailedScheduling / Insufficient cpu' event to confirm. The fix is to ensure the Cluster Autoscaler is enabled on the node pool (`az aks update --enable-cluster-autoscaler --min-count X --max-count Y`) so Kubernetes automatically provisions new nodes when Pods can't be scheduled on existing capacity — HPA scales Pod count, but Cluster Autoscaler scales node count, and both need to be configured together for the system to truly scale end-to-end under load.

TF

CHAPTER 22 · INFRASTRUCTURE AS CODE

Infrastructure as Code — Terraform with Azure

Terraform with the azurerm provider — state, modules, CI/CD workflow.

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp that lets you define Azure resources — VMs, VNETs, Storage Accounts, AKS clusters, everything covered in this document — in declarative configuration files, then plan and apply changes in a repeatable, version-controlled, peer-reviewed way, instead of clicking through the Azure Portal.

💡 Layman Explanation

Clicking through the Azure Portal to build infrastructure is like building furniture from memory, one piece at a time, with no instruction manual — hard to repeat exactly, hard to hand off to someone else, and easy to forget a step. Terraform is the IKEA instruction manual: a precise, written blueprint that anyone (or any automated pipeline) can follow to build the exact same result every single time, in any region, as many times as needed.

22.1 Core Terraform Concepts

📝 Theory & Key Points

- A **Provider** (e.g. azurerm) is the plugin that translates Terraform's generic language into actual Azure API calls.
- A **Resource** block declares one piece of infrastructure (a VM, a VNET). Terraform computes the difference between your declared configuration and the real, current state of the world, and only changes what's actually different.
- The **State File** (terraform.tfstate) is Terraform's record of what it believes exists and their exact IDs/attributes — critical for tracking. For team use, state must be stored remotely (e.g. an Azure Storage Account container with state locking) rather than on a single laptop, to prevent conflicting concurrent applies and to avoid losing the state entirely.
- The core workflow is always: **terraform init** (download providers) → **terraform plan** (preview changes, no-op) → **terraform apply** (execute changes) → **terraform destroy** (tear down).

22.2 Example Terraform Configuration — Azure VM + VNET

HCL

```
# providers.tf — configure remote state + the azurerm provider
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 3.100" }
  }
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "sttfstatedevops001"
    container_name       = "tfstate"
    key                  = "devopslab.terraform.tfstate"
  }
}

provider "azurerm" { features {} }

# main.tf — resource group, VNET, subnet, NSG, and a VM
resource "azurerm_resource_group" "rg" {
  name     = "rg-devops-lab-tf"
  location = "Central India"
}

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-tf"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "web" {
  name                 = "snet-web"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                = "vm-tf-web01"
  resource_group_name   = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  size                = "Standard_B2s"
  admin_username      = "veera"
  network_interface_ids = [azurerm_network_interface.nic.id]

  admin_ssh_key {
    username   = "veera"
    public_key = file("~/.ssh/id_rsa.pub")
  }
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}
```

BASH

```
# The standard Terraform workflow
terraform init            # download the azurerm provider + configure remote state
terraform fmt              # auto-format the .tf files consistently
terraform validate         # check syntax and internal consistency
terraform plan -out=tfplan # preview exactly what will change
terraform apply tfplan     # execute the plan, create/modify resources
terraform state list       # see everything Terraform is tracking
terraform destroy          # tear everything down (use with extreme caution)
```

22.3 State Management & Team Workflow

📝 Theory & Key Points

- **Remote State with Locking:** storing state in an Azure Storage Account (with blob leasing acting as a lock) prevents two engineers from running `terraform apply` simultaneously and corrupting the state.
- **Workspaces** or, more commonly in real teams, separate state files per environment (dev/staging/prod) via distinct backend keys, keep environments fully isolated so a mistake in dev can never touch prod's state.
- **Modules** let you package reusable, parameterised infrastructure patterns (e.g. a standard '3-tier web app' module) and reuse them across projects instead of copy-pasting HCL.
- Terraform should always run inside a CI/CD pipeline (Azure Pipelines/GitHub Actions) for production changes — plan on every Pull Request for visible review, apply only after merge/approval, never applied ad-hoc from an engineer's laptop against production.

⚠️ Important Warning

- Never manually edit resources created by Terraform directly in the Azure Portal — this causes 'configuration drift' where the real world no longer matches the state file, leading to unpredictable behaviour on the next apply.
- Never commit the .tfstate file or any .tfvars file containing secrets to Git — state files can contain sensitive data in plaintext (e.g. generated passwords); use remote state + Key Vault-sourced variables instead.

🎯 Scenario-Based Interview Questions

Q: Two engineers on your team both ran 'terraform apply' within a minute of each other against the same production environment, and the state file is now corrupted/inconsistent. How should the team be set up to prevent this, and how do you recover? Prevention: configure a remote backend (Azure Storage Account) for state, which supports locking — Azure Blob Storage leasing means Terraform automatically blocks a second concurrent apply until the first one finishes, making this scenario structurally impossible rather than just discouraged by process. Enforce that all applies to production happen only through a CI/CD pipeline (which naturally serialises runs) rather than from individual laptops. Recovery: use `terraform state list` and `terraform plan` to see what Terraform currently believes exists versus the real Azure resources, use `terraform state pull` to inspect the raw state, and if genuinely corrupted, restore the previous state file from the Storage Account's blob versioning/soft-delete (which is why versioning should always be enabled on the state storage account) before carefully reconciling any resources created during the conflicting applies with `terraform import`.

Azure Cloud & DevOps — Master Study Guide, compiled from the complete Azure Cloud & DevOps Study Notes by Veera Sir, covering Cloud Fundamentals, Linux Basics, Compute (VMs, Load Balancer, Auto Scaling), Storage Services, Networking (VNET), Object Storage (Blob), Identity & Access (Microsoft Entra ID), Databases (Azure SQL), Serverless (App Service & Functions), CI/CD Pipelines (Azure DevOps), Orchestration (AKS), and Infrastructure as Code (Terraform).

Theory · Diagrams · Commands · Scenario-Based Interview Q&A